



POLITECNICO
MILANO 1863

Sorting on Two-Dimensional Processor Arrays

Hans Werner Lang

Conti Alessandro

24/09/2025

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ $3n$ -Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑧ Conclusion
- ⑨ References

1 Introduction

2 LS3-Sort

- Algorithms
- Correctness
- Analysis
- Example

3 4-Way Mergesort

- Algorithms
- Correctness
- Analysis
- Example

4 Rotatesort

- Algorithms
- Correctness
- Analysis
- Example

5 $3n$ -Sort of Schnorr and Shamir

- Algorithm
- Correctness
- Analysis
- Example

6 2D Odd-Even Transposition Sort

- Algorithm
- Correctness
- Analysis
- Example

7 Shearsort

- Algorithm
- Correctness
- Analysis
- Example

8 Conclusion

9 References

The definition of the sorting problem has to be generalized in order to cope with two-dimensional arrays (and moreover, arbitrary arrangements) of data.

Definition (Data Record)

A **Data Record** is a mapping

$$a : J \rightarrow A$$

Where $J = \{0, \dots, n-1\} \times \{0, \dots, n-1\}$ is a finite index set representing a two-dimensional array of size $n \times n$ and A is a set of data elements equipped with some order relation $\preceq$.

We denote the elements of the record as

$$A = a_{i,j} \big|_{(i,j) \in J}$$

Example (Data Record)

Suppose:

- $J = \{0, 1\} \times \{0, 1\}$;
- $A = \mathbb{N}$;
- $\preceq = \leq$;
- $a : J \rightarrow \mathbb{N}$, that specifies a value for each index

$$a(0, 0) = 4 \quad a(0, 1) = 2 \quad a(1, 0) = 7 \quad a(1, 1) = 1$$

The data record can be represented as

$$A = \begin{bmatrix} a_{0,0} & a_{0,1} \\ a_{1,0} & a_{1,1} \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 7 & 1 \end{bmatrix}$$

Definition (Sorting Order)

A **Sorting Order** is a bijective mapping

$$\rho : J \rightarrow \{0, \dots, |J| - 1\}$$

- *row-major*: $\rho(i, j) = i \cdot n + j$
- *snake-like*: $\rho(i, j) = \begin{cases} i \cdot n + j & i \text{ even} \\ i \cdot n + |J| - 1 - j & i \text{ odd} \end{cases}$

Example (Sorting Order)

- Row-major order

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

- Snake-like order

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 8 & 7 & 6 & 5 \\ 9 & 10 & 11 & 12 \\ 16 & 15 & 14 & 13 \end{bmatrix}$$

Definition (Generalized Sorting Problem)

The **Generalized Sorting Problem** is defined to reorder a data record $a_{i,j} \Big|_{(i,j) \in J}$ to a data record $a_{\varphi(i,j)} \Big|_{(i,j) \in J}$ such that:

$$\forall (i_1, i_2), (j_1, j_2) \in J : a_{\varphi(i_1, i_2)} \Big|_{(i,j) \in J} \preceq a_{\varphi(i, j)} \Big|_{(j_1, j_2) \in J} \\ \text{for } \rho(i_1, i_2) \prec \rho(j_1, j_2)$$

The function φ is a permutation of the index set J .

Example (Generalized Sorting Problem)

Suppose:

- $J = \{0, 1\} \times \{0, 1\}$;
- $A = \mathbb{N}$;
- $\preceq = \leq$;
- $A = \begin{bmatrix} a_{0,0} & a_{0,1} \\ a_{1,0} & a_{1,1} \end{bmatrix} = \begin{bmatrix} 7 & 3 \\ 4 & 9 \end{bmatrix}$;
- $\rho(i, j) = i \cdot n + j$;

- $\varphi : J \rightarrow J$ the permutation function

$$\varphi((0, 0)) = (0, 1) \quad \varphi((0, 1)) = (1, 0)$$

$$\varphi((1, 0)) = (0, 0) \quad \varphi((1, 1)) = (1, 1)$$

Now the sorted data record is

$$A_{\varphi} = \begin{bmatrix} 3 & 4 \\ 7 & 9 \end{bmatrix}$$

Theorem (0-1-Principle [1])

If a sorting network sorts every sequence of 0's and 1's, then it sorts every arbitrary sequence of values.

- 1 Introduction
- 2 LS3-Sort**
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 3 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 4 Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 5 $3n$ -Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 6 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 7 Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 8 Conclusion
- 9 References

The **LS3-Sort** [2] is a very simple algorithm for sorting on a two-dimensional mesh.

The algorithm is based on a merge algorithm that merges 4 sorted $\frac{k}{2} \times \frac{k}{2}$ -arrays to one sorted $k \times k$ -array.

The sorting direction is the snake.

The merge algorithm use of the basic operations *shuffle* and *oets*.

- *Shuffle*: a deck of cards can be mixed by exactly interleaving its two halves;
- *oets*: stands for one step of odd-even transposition sort.

Definition (Odd-Even Transposition Sort [1])

The **Odd-Even Transposition Sort** for n input data consists of n comparators stages. In each stage, either all inputs at odd index positions or all inputs at even index positions are compared with their neighbours. Odd and even stages alternate. The number of comparators is $n \frac{n-1}{2}$.

- 1 Introduction
- 2 **LS3-Sort**
 - **Algorithms**
 - Correctness
 - Analysis
 - Example
- 3 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 4 Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 5 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 6 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 7 Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 8 Conclusion
- 9 References

Algorithm 1 LS3-sort Algorithm

Input:

$M \in \mathbb{Z}^{n \times n}$: unsorted matrix

Output:

$M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```
1 function LS3SORT(M)
2   if  $n > 1$  then
3     subNW  $\leftarrow$  GETSUBMATRIX(M, "NW")
4     subNW'  $\leftarrow$  LS3SORT(subNW)
5     subNE  $\leftarrow$  GETSUBMATRIX(M, "NE")
6     subNE'  $\leftarrow$  LS3SORT(subNE)
7     subSW  $\leftarrow$  GETSUBMATRIX(M, "SW")
8     subSW'  $\leftarrow$  LS3SORT(subSW)
9     subSE  $\leftarrow$  GETSUBMATRIX(M, "SE")
10    subSE'  $\leftarrow$  LS3SORT(subSE)
11    M'  $\leftarrow$  ASSEMBLE(subNW', subNE', subSW', subSE')
12    M'  $\leftarrow$  LS3MERGE(M')
13    return M'
14  else
15    return M
16  end if
17 end function
```

Algorithm 2 LS3-merge Algorithm

Input: $M \in \mathbb{Z}^{k \times k}$: unsorted matrix, whose submatrices of size $\frac{k}{2} \times \frac{k}{2}$ are snake-sorted**Output:** $M' \in \mathbb{Z}^{k \times k}$: snake-sorted matrix

```
1 function LS3MERGE(M)
2   for  $i \leftarrow 0$  to  $k - 1$  do
3      $M_{\text{tmp}}[i, :] \leftarrow \text{SHUFFLE}(M[i, :])$ 
4   end for
5   for  $j \leftarrow 0$  to  $k - 1$  step 2 do
6      $\langle M_{\text{tmp}}[:, j], M_{\text{tmp}}[:, j + 1] \rangle \leftarrow \text{SORTSNAKELIKE}(M_{\text{tmp}}[:, j], M_{\text{tmp}}[:, j + 1])$ 
7   end for
8    $M' \leftarrow \text{OETS}(M_{\text{tmp}})$ 
9   return  $M'$ 
10 end function
```

- ① Introduction
- ② **LS3-Sort**
 - Algorithms
 - **Correctness**
 - Analysis
 - Example
- ③ 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑧ Conclusion
- ⑨ References

Our initial situation is an array filled with 0 and 1, whose four subarrays are sorted in a snake-like direction.

Each subarray contains a certain number of full rows, labelled in the figure as a , b , c , and d , and possibly an incomplete row.

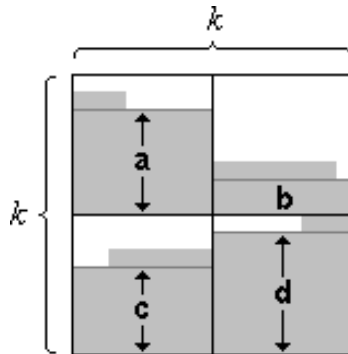


Figure: Initial situation

After the SHUFFLE operation, in every double column there are $a + b + c + d$ ones that stem from the full rows plus at most four more ones from the incomplete rows.

Take the first row of subNW and the first row of subNE, join them to form the first row of the whole matrix. Then the second row of subNW and the second row of subNE, etc. Then do the same for subSW and subSE.

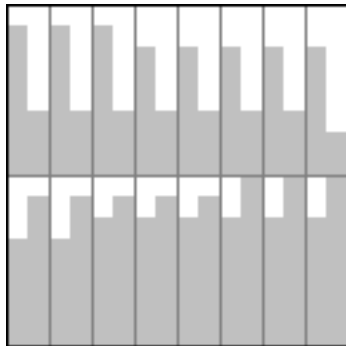


Figure: After Shuffle

Each double column is sorted in snake-like direction.

$a + b + c + d$ **is even**.

Then the 1s from the full rows form $\frac{a+b+c+d}{2}$ full rows.

Additionally, in each double column there are at most 4 1s from the incomplete rows. These 1s form an unsorted zone consisting of at most two rows.

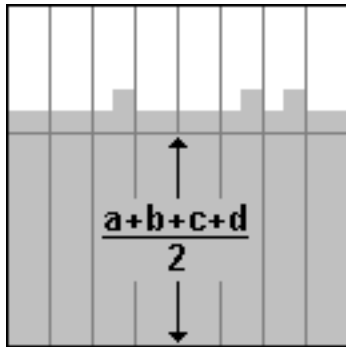


Figure: After Sort Double Column

Each double column is sorted in snake-like direction.

$a + b + c + d$ **is odd**.

Then the 1s from the full rows form a step in each double column.

However, this is only possible if all incomplete rows start from the same side. Since the sorting direction is the snake this implies that the numbers a, b, c, d are all even or all odd. But then their sum cannot be odd.

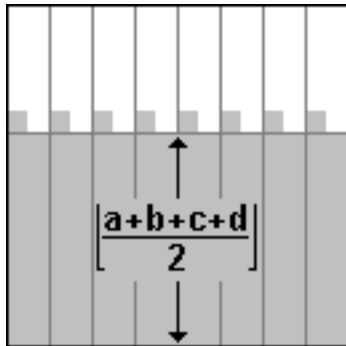


Figure: After Sort Double Column

Apply OETS and the unsorted zone is sorted.

The unsorted zone is at most $2k$, this means that requires $2k$ steps to sort that.

- 1 Introduction
- 2 **LS3-Sort**
 - Algorithms
 - Correctness
 - **Analysis**
 - Example
- 3 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 4 Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 5 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 6 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 7 Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 8 Conclusion
- 9 References

Given a matrix $k \times k$. The number of steps and comparators are:

- SHUFFLE along the rows: $\frac{1}{2}k$ steps and no comparators, as the shuffle is merely a permutation;
- SORTSNAKELIKE to each double column: $2k$ steps and $\frac{k}{2} \frac{2k(2k-1)}{2}$ comparators;
- apply $2k$ OETS step to the snake: $2k$ steps and $2k \frac{k^2}{2}$ comparators.

Given a matrix $n \times n$. The number of steps are:

$$\begin{aligned} T(n^2) &= \left(n + \frac{n}{2} + \frac{n}{4} + \dots + 2 \right) 4.5 = \\ &= \sum_{i=0}^{\log_2 n - 1} \frac{9}{2} \frac{n}{2^i} = \\ &= 9n - 9 \end{aligned}$$

and the number of comparators are:

$$\begin{aligned} C(n^2) &= \sum_{i=0}^{\log_2 n - 1} 4^i \frac{\left(\frac{n}{2^i}\right)^2 (4\frac{n}{2^i} - 1)}{2} = \\ &= 4n^3 - \frac{1}{2}n^2 \log_2 n - 4n^2 \end{aligned}$$

- 1 Introduction
- 2 **LS3-Sort**
 - Algorithms
 - Correctness
 - Analysis
 - **Example**
- 3 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 4 Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 5 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 6 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 7 Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 8 Conclusion
- 9 References

Algorithm 3 LS3-sort Algorithm**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```

1 function LS3SORT(M)
2   if  $n > 1$  then
3     subNW  $\leftarrow$  GETSUBMATRIX(M, "NW")
4     subNW'  $\leftarrow$  LS3SORT(subNW)
5     subNE  $\leftarrow$  GETSUBMATRIX(M, "NE")
6     subNE'  $\leftarrow$  LS3SORT(subNE)
7     subSW  $\leftarrow$  GETSUBMATRIX(M, "SW")
8     subSW'  $\leftarrow$  LS3SORT(subSW)
9     subSE  $\leftarrow$  GETSUBMATRIX(M, "SE")
10    subSE'  $\leftarrow$  LS3SORT(subSE)
11    M'  $\leftarrow$ 
    ASSEMBLE(subNW', subNE', subSW', subSE')
12    M'  $\leftarrow$  LS3MERGE(M')
13    return M'
14  else
15    return M
16  end if
17 end function

```

Initial Situation

$$\begin{bmatrix} 0 & 1 & 7 & 12 \\ 4 & 5 & 9 & 8 \\ 13 & 6 & 15 & 14 \\ 3 & 2 & 11 & 10 \end{bmatrix}$$

Apply merge to the submatrixNW (algorithm 5)

$$\begin{bmatrix} 0 & 1 \\ 4 & 5 \end{bmatrix}$$

After shuffle (algorithm 4)

$$\begin{bmatrix} 0 & 1 \\ 5 & 4 \end{bmatrix}$$

After sort the double columns (algorithm 4)

$$\begin{bmatrix} 0 & 1 \\ 4 & 5 \end{bmatrix}$$

After oets (algorithm 4)

$$\begin{bmatrix} 0 & 1 \\ 5 & 4 \end{bmatrix}$$

Algorithm 4 LS3-merge Algorithm**Input:**

$M \in \mathbb{Z}^{k \times k}$: unsorted matrix, whose submatrices
of size $\frac{k}{2} \times \frac{k}{2}$ are snake-sorted

Output:

$M' \in \mathbb{Z}^{k \times k}$: snake-sorted matrix

```

1 function LS3MERGE(M)
2   for  $i \leftarrow 0$  to  $k - 1$  do
3      $M_{\text{tmp}}[i, :] \leftarrow \text{SHUFFLE}(M[1, :])$ 
4   end for
5   for  $j \leftarrow 0$  to  $k - 1$  step 2 do
6      $\langle M_{\text{tmp}}[:, j], M_{\text{tmp}}[:, j + 1] \rangle \leftarrow$ 
      SORTSNAKELIKE( $M_{\text{tmp}}[:, j], M_{\text{tmp}}[:, j + 1]$ )
7   end for
8    $M' \leftarrow \text{OETS}(M_{\text{tmp}})$ 
9   return  $M'$ 
10 end function

```

Algorithm 5 LS3-sort Algorithm**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```

1 function LS3SORT(M)
2   if  $n > 1$  then
3     subNW  $\leftarrow$  GETSUBMATRIX(M, "NW")
4     subNW'  $\leftarrow$  LS3SORT(subNW)
5     subNE  $\leftarrow$  GETSUBMATRIX(M, "NE")
6     subNE'  $\leftarrow$  LS3SORT(subNE)
7     subSW  $\leftarrow$  GETSUBMATRIX(M, "SW")
8     subSW'  $\leftarrow$  LS3SORT(subSW)
9     subSE  $\leftarrow$  GETSUBMATRIX(M, "SE")
10    subSE'  $\leftarrow$  LS3SORT(subSE)
11    M'  $\leftarrow$ 
    ASSEMBLE(subNW', subNE', subSW', subSE')
12    M'  $\leftarrow$  LS3MERGE(M')
13    return M'
14  else
15    return M
16  end if
17 end function

```

Apply merge to the submatrixNE (algorithm 5)

$$\begin{bmatrix} 7 & 12 \\ 9 & 8 \end{bmatrix}$$

After shuffle (algorithm 6)

$$\begin{bmatrix} 7 & 12 \\ 8 & 9 \end{bmatrix}$$

After sort the double columns (algorithm 6)

$$\begin{bmatrix} 7 & 8 \\ 9 & 12 \end{bmatrix}$$

After oets (algorithm 6)

$$\begin{bmatrix} 7 & 8 \\ 12 & 9 \end{bmatrix}$$

Algorithm 6 LS3-merge Algorithm

Input:

$M \in \mathbb{Z}^{k \times k}$: unsorted matrix, whose submatrices
of size $\frac{k}{2} \times \frac{k}{2}$ are snake-sorted

Output:

$M' \in \mathbb{Z}^{k \times k}$: snake-sorted matrix

```

1 function LS3MERGE(M)
2   for  $i \leftarrow 0$  to  $k - 1$  do
3      $M_{\text{tmp}}[i, :] \leftarrow \text{SHUFFLE}(M[1, :])$ 
4   end for
5   for  $j \leftarrow 0$  to  $k - 1$  step 2 do
6      $\langle M_{\text{tmp}}[:, j], M_{\text{tmp}}[:, j + 1] \rangle \leftarrow$ 
      SORTSNAKELIKE( $M_{\text{tmp}}[:, j], M_{\text{tmp}}[:, j + 1]$ )
7   end for
8    $M' \leftarrow \text{OETS}(M_{\text{tmp}})$ 
9   return  $M'$ 
10 end function

```

Algorithm 7 LS3-sort Algorithm**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```

1 function LS3SORT(M)
2   if  $n > 1$  then
3     subNW  $\leftarrow$  GETSUBMATRIX(M, "NW")
4     subNW'  $\leftarrow$  LS3SORT(subNW)
5     subNE  $\leftarrow$  GETSUBMATRIX(M, "NE")
6     subNE'  $\leftarrow$  LS3SORT(subNE)
7     subSW  $\leftarrow$  GETSUBMATRIX(M, "SW")
8     subSW'  $\leftarrow$  LS3SORT(subSW)
9     subSE  $\leftarrow$  GETSUBMATRIX(M, "SE")
10    subSE'  $\leftarrow$  LS3SORT(subSE)
11    M'  $\leftarrow$ 
        ASSEMBLE(subNW', subNE', subSW', subSE')
12    M'  $\leftarrow$  LS3MERGE(M')
13    return M'
14  else
15    return M
16  end if
17 end function

```

Apply merge to the submatrixSW (algorithm 7)

$$\begin{bmatrix} 13 & 6 \\ 3 & 2 \end{bmatrix}$$

After shuffle (algorithm 8)

$$\begin{bmatrix} 13 & 6 \\ 2 & 3 \end{bmatrix}$$

After sort the double columns (algorithm 8)

$$\begin{bmatrix} 2 & 3 \\ 6 & 13 \end{bmatrix}$$

After oets (algorithm 8)

$$\begin{bmatrix} 2 & 3 \\ 13 & 6 \end{bmatrix}$$

Algorithm 8 LS3-merge Algorithm**Input:**

$M \in \mathbb{Z}^{k \times k}$: unsorted matrix, whose submatrices
of size $\frac{k}{2} \times \frac{k}{2}$ are snake-sorted

Output:

$M' \in \mathbb{Z}^{k \times k}$: snake-sorted matrix

```

1 function LS3MERGE(M)
2   for  $i \leftarrow 0$  to  $k - 1$  do
3      $M_{\text{tmp}}[i, :] \leftarrow \text{SHUFFLE}(M[1, :])$ 
4   end for
5   for  $j \leftarrow 0$  to  $k - 1$  step 2 do
6      $\langle M_{\text{tmp}}[:, j], M_{\text{tmp}}[:, j + 1] \rangle \leftarrow$ 
      SORTSNAKELIKE( $M_{\text{tmp}}[:, j], M_{\text{tmp}}[:, j + 1]$ )
7   end for
8    $M' \leftarrow \text{OETS}(M_{\text{tmp}})$ 
9   return  $M'$ 
10 end function

```

Algorithm 9 LS3-sort Algorithm**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```

1 function LS3SORT(M)
2   if  $n > 1$  then
3     subNW  $\leftarrow$  GETSUBMATRIX(M, "NW")
4     subNW'  $\leftarrow$  LS3SORT(subNW)
5     subNE  $\leftarrow$  GETSUBMATRIX(M, "NE")
6     subNE'  $\leftarrow$  LS3SORT(subNE)
7     subSW  $\leftarrow$  GETSUBMATRIX(M, "SW")
8     subSW'  $\leftarrow$  LS3SORT(subSW)
9     subSE  $\leftarrow$  GETSUBMATRIX(M, "SE")
10    subSE'  $\leftarrow$  LS3SORT(subSE)
11    M'  $\leftarrow$ 
        ASSEMBLE(subNW', subNE', subSW', subSE')
12    M'  $\leftarrow$  LS3MERGE(M')
13    return M'
14  else
15    return M
16  end if
17 end function

```

Apply merge to the submatrixSE (algorithm 9)

$$\begin{bmatrix} 15 & 14 \\ 11 & 10 \end{bmatrix}$$

After shuffle (algorithm 10)

$$\begin{bmatrix} 15 & 14 \\ 10 & 11 \end{bmatrix}$$

After sort the double columns (algorithm 10)

$$\begin{bmatrix} 10 & 11 \\ 14 & 15 \end{bmatrix}$$

After oets (algorithm 10)

$$\begin{bmatrix} 10 & 11 \\ 15 & 14 \end{bmatrix}$$

Algorithm 10 LS3-merge Algorithm

Input:

$M \in \mathbb{Z}^{k \times k}$: unsorted matrix, whose submatrices
of size $\frac{k}{2} \times \frac{k}{2}$ are snake-sorted

Output:

$M' \in \mathbb{Z}^{k \times k}$: snake-sorted matrix

```

1 function LS3MERGE(M)
2   for  $i \leftarrow 0$  to  $k - 1$  do
3      $M_{\text{tmp}}[i, :] \leftarrow \text{SHUFFLE}(M[1, :])$ 
4   end for
5   for  $j \leftarrow 0$  to  $k - 1$  step 2 do
6      $\langle M_{\text{tmp}}[:, j], M_{\text{tmp}}[:, j + 1] \rangle \leftarrow$ 
      SORTSNAKELIKE( $M_{\text{tmp}}[:, j], M_{\text{tmp}}[:, j + 1]$ )
7   end for
8    $M' \leftarrow \text{OETS}(M_{\text{tmp}})$ 
9   return  $M'$ 
10 end function

```

Algorithm 11 LS3-sort Algorithm**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```

1 function LS3SORT(M)
2   if  $n > 1$  then
3     subNW  $\leftarrow$  GETSUBMATRIX(M, "NW")
4     subNW'  $\leftarrow$  LS3SORT(subNW)
5     subNE  $\leftarrow$  GETSUBMATRIX(M, "NE")
6     subNE'  $\leftarrow$  LS3SORT(subNE)
7     subSW  $\leftarrow$  GETSUBMATRIX(M, "SW")
8     subSW'  $\leftarrow$  LS3SORT(subSW)
9     subSE  $\leftarrow$  GETSUBMATRIX(M, "SE")
10    subSE'  $\leftarrow$  LS3SORT(subSE)
11    M'  $\leftarrow$  ASSEMBLE(subNW', subNE', subSW', subSE')
12    M'  $\leftarrow$  LS3MERGE(M')
13    return M'
14  else
15    return M
16  end if
17 end function

```

Apply merge to the matrix, whose submatrices are sorted in snake-like direction (algorithm 11)

$$\begin{bmatrix} 0 & 1 & 7 & 8 \\ 5 & 4 & 12 & 9 \\ 2 & 3 & 10 & 11 \\ 13 & 6 & 15 & 14 \end{bmatrix}$$

Algorithm 12 LS3-merge Algorithm**Input:**

$M \in \mathbb{Z}^{k \times k}$: unsorted matrix, whose submatrices
of size $\frac{k}{2} \times \frac{k}{2}$ are snake-sorted

Output:

$M' \in \mathbb{Z}^{k \times k}$: snake-sorted matrix

```

1 function LS3MERGE(M)
2   for  $i \leftarrow 0$  to  $k - 1$  do
3      $M_{\text{tmp}}[i, :] \leftarrow \text{SHUFFLE}(M[1, :])$ 
4   end for
5   for  $j \leftarrow 0$  to  $k - 1$  step 2 do
6      $\langle M_{\text{tmp}}[:, j], M_{\text{tmp}}[:, j + 1] \rangle \leftarrow$ 
      SORTSNAKELIKE( $M_{\text{tmp}}[:, j], M_{\text{tmp}}[:, j + 1]$ )
7   end for
8    $M' \leftarrow \text{OETS}(M_{\text{tmp}})$ 
9   return  $M'$ 
10 end function

```

After shuffle (algorithm 12)

$$\begin{bmatrix} 0 & 1 & 7 & 8 \\ 9 & 12 & 4 & 5 \\ 2 & 3 & 10 & 11 \\ 14 & 15 & 6 & 13 \end{bmatrix}$$

After sort the double columns (algorithm 12)

$$\begin{bmatrix} 0 & 1 & 4 & 5 \\ 2 & 3 & 6 & 7 \\ 9 & 12 & 8 & 10 \\ 14 & 15 & 11 & 13 \end{bmatrix}$$

After oets (algorithm 12)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 7 & 6 & 5 & 4 \\ 8 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

- 1 Introduction
- 2 LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 3 4-Way Mergesort**
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 4 Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 5 $3n$ -Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 6 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 7 Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 8 Conclusion
- 9 References

Definition (Roughly Sorted Array)

A $m \times n$ -array of data is **Roughly Sorted**, if sorting of the rows suffices to sort the array completely

In a roughly sorted array each data element is already in its proper row.

Example (Roughly Sorted Array)

$$\begin{bmatrix} 3 & 0 & 2 & 1 \\ 4 & 5 & 6 & 7 \\ 11 & 10 & 9 & 8 \\ 13 & 12 & 15 & 14 \end{bmatrix}$$

The idea of 4-way mergesort [5] is to merge 4 roughly sorted $\frac{k}{2} \times \frac{k}{2}$ -arrays to one roughly sorted $k \times k$ -array.

The sorting direction is row-major.

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ **4-Way Mergesort**
 - **Algorithms**
 - Correctness
 - Analysis
 - Example
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ $3n$ -Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑧ Conclusion
- ⑨ References

Algorithm 13 4-Way Mergesort Algorithm

Input: $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: row-major sorted matrix

```
1 function FOURWAYMERGESORT(M)
2    $M' \leftarrow \text{ROUGH SORT}(M)$ 
3   for  $i \leftarrow 0$  to  $n - 1$  do
4      $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
5   end for
6   return  $M'$ 
7 end function
```

Algorithm 14 Roughsort Algorithm

Input: $M \in \mathbb{Z}^{k \times k}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{k \times k}$: roughly-sorted matrix

```
1 function ROUGHSORT(M)
2   if  $k > 1$  then
3     subNW  $\leftarrow$  GETSUBMATRIX(M, "NW")
4     subNW'  $\leftarrow$  ROUGHSORT(subNW)
5     subNE  $\leftarrow$  GETSUBMATRIX(M, "NE")
6     subNE'  $\leftarrow$  ROUGHSORT(subNE)
7     subSW  $\leftarrow$  GETSUBMATRIX(M, "SW")
8     subSW'  $\leftarrow$  ROUGHSORT(subSW)
9     subSE  $\leftarrow$  GETSUBMATRIX(M, "SE")
10    subSE'  $\leftarrow$  ROUGHSORT(subSE)
11    M'  $\leftarrow$  ASSEMBLE(subNW', subNE', subSW', subSE')
12    M'  $\leftarrow$  FOURWAYMERGE(M')
13    return M'
14  else
15    return M
16  end if
17 end function
```

Algorithm 15 4-Way Merge Algorithm

Input: $M \in \mathbb{Z}^{k \times k}$: unsorted matrix, whose submatrices of size $\frac{k}{2} \times \frac{k}{2}$ are roughly-sorted**Output:** $M' \in \mathbb{Z}^{k \times k}$: roughly-sorted matrix

```

1 function FOURWAYMERGE(M)
2   for  $i \leftarrow 0$  to  $k - 1$  do
3     if  $i < \frac{k}{2}$  then
4        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"ASC"})$ 
5     else
6        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"DESC"})$ 
7     end if
8   end for

```

```

9   for  $j \leftarrow 0$  to  $k - 1$  do
10      $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
11   end for
12   for  $i \leftarrow 0$  to  $k - 1$  do
13     if  $i \bmod 2 = 1$  then
14        $M'[i, :] \leftarrow \text{SORT}(M_{\text{tmp}}[i, :], \text{"ASC"})$ 
15     else
16        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"DESC"})$ 
17     end if
18   end for
19   for  $j \leftarrow 0$  to  $k - 1$  do
20      $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
21   end for
22 end function

```

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ **4-Way Mergesort**
 - Algorithms
 - **Correctness**
 - Analysis
 - Example
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ $3n$ -Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑧ Conclusion
- ⑨ References

Our initial situation is an array filled with 0 and 1, whose four subarrays are roughly sorted.

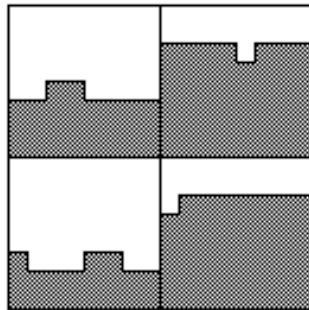


Figure: Initial situation

Sort the rows of the subarrays:

- ascending in the upper subarrays;
- descending in the lower subarrays.

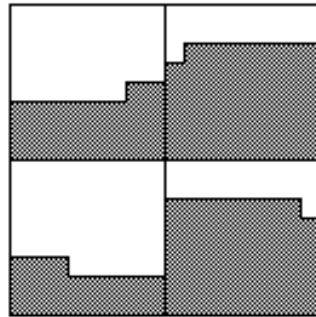


Figure: Sort the Rows of the Subarrays

Sort the column in ascending order.
Now, we have 2 roughly sorted $k \times \frac{k}{2}$ -arrays.

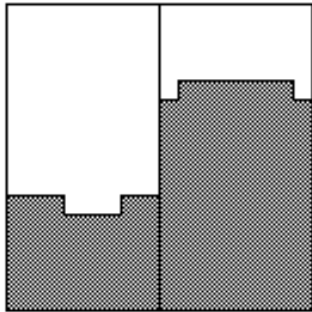


Figure: Sort the Columns

Sort the rows:

- odd rows ascending order;
- even rows descending order.

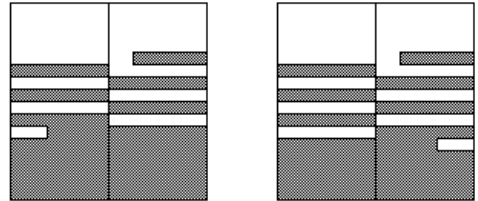


Figure: Sort the Rows

Sort the columns in ascending order.

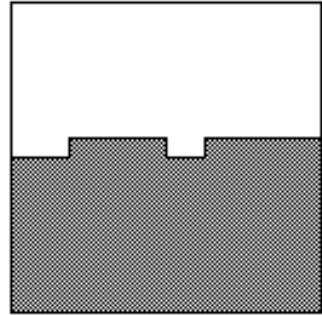


Figure: Sort the Columns

Sort the rows in ascending order.

- 1 Introduction
- 2 LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 3 4-Way Mergesort**
 - Algorithms
 - Correctness
 - **Analysis**
 - Example
- 4 Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 5 $3n$ -Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 6 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 7 Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 8 Conclusion
- 9 References

Given a matrix $k \times k$. The number of steps and comparators are:

- *sort the rows of the subarrays*: $\frac{1}{2}k$ steps and $2k \frac{\frac{k}{2}(\frac{k}{2}-1)}{2}$ comparators;
- *sort the columns*: k steps and $k \frac{k(k-1)}{2}$ comparators;
- *sort the rows*: k steps and $k \frac{k(k-1)}{2}$ comparators;
- *sort the columns*: $\frac{1}{2}k$ steps and $k \frac{k(k-1)}{2}$, since each column consists of two sorted subsequences shuffled into each other.

Given a matrix $n \times n$. The number of steps to sort a matrix are

$$\begin{aligned} T(n^2) &= \left(n + \frac{n}{2} + \frac{n}{4} + \dots + 1 \right) 3 = \\ &= \sum_{i=0}^{\log_2 n - 1} 3 \frac{n}{2^i} = \\ &= 6n - 6 \end{aligned}$$

and the number of comparator are

$$\begin{aligned} C(n^2) &= \sum_{i=0}^{\log_2 n - 1} 4^i \frac{\left(\frac{n}{2^i}\right)^2 \left(7 \frac{n}{2^i} - 8\right)}{4} = \\ &= \frac{7}{2} n^3 - 2n^2 \log_2 n - \frac{7}{2} n^2 \end{aligned}$$

Given a matrix $n \times n$. The number of steps to sort a matrix are

$$\begin{aligned} T(n^2) &= \left(n + \frac{n}{2} + \frac{n}{4} + \dots + 1 \right) 3 + n = \\ &= \sum_{i=0}^{\log_2 n - 1} 3 \frac{n}{2^i} + n = \\ &= 7n - 6 \end{aligned}$$

and the number of comparator are

$$\begin{aligned} C(n^2) &= \sum_{i=0}^{\log_2 n - 1} 4^i \frac{\left(\frac{n}{2^i}\right)^2 (7 \frac{n}{2^i} - 8)}{4} + n \frac{n(n-1)}{2} = \\ &= 4n^3 - 2n^2 \log_2 n - 4n^2 \end{aligned}$$

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ **4-Way Mergesort**
 - Algorithms
 - Correctness
 - Analysis
 - **Example**
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ $3n$ -Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑧ Conclusion
- ⑨ References

Algorithm 16 4-Way Mergesort Algorithm

Input: $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: row-major sorted matrix

```
1 function FOURWAYMERGESORT(M)
2    $M' \leftarrow \text{ROUGHSORT}(M)$ 
3   for  $i \leftarrow 0$  to  $n - 1$  do
4      $M[i, :] \leftarrow \text{SORT}(M[i, :], \text{"ASC"})$ 
5   end for
6   return  $M'$ 
7 end function
```

Initial situation

$$\begin{bmatrix} 0 & 1 & 7 & 12 \\ 4 & 5 & 9 & 8 \\ 13 & 6 & 15 & 14 \\ 3 & 2 & 11 & 10 \end{bmatrix}$$

Algorithm 17 Roughsort Algorithm

Input: $M \in \mathbb{Z}^{k \times k}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{k \times k}$: roughly-sorted matrix

```

1 function ROUGHSORT(M)
2   if  $k > 1$  then
3     subNW  $\leftarrow$  GETSUBMATRIX(M, "NW")
4     subNW'  $\leftarrow$  ROUGHSORT(subNW)
5     subNE  $\leftarrow$  GETSUBMATRIX(M, "NE")
6     subNE'  $\leftarrow$  ROUGHSORT(subNE)
7     subSW  $\leftarrow$  GETSUBMATRIX(M, "SW")
8     subSW'  $\leftarrow$  ROUGHSORT(subSW)
9     subSE  $\leftarrow$  GETSUBMATRIX(M, "SE")
10    subSE'  $\leftarrow$  ROUGHSORT(subSE)
11    M'  $\leftarrow$ 
        ASSEMBLE(subNW', subNE', subSW', subSE')
12    M'  $\leftarrow$  FOURWAYMERGE(M')
13    return M'
14  else
15    return M
16  end if
17 end function

```

Apply merge to the submatrixNW

$$\begin{bmatrix} 0 & 1 \\ 4 & 5 \end{bmatrix}$$

Algorithm 18 4-Way Merge Algorithm**Input:** $M \in \mathbb{Z}^{k \times k}$: unsorted matrix, whose submatrices of size $\frac{k}{2} \times \frac{k}{2}$ are roughly-sorted**Output:** $M' \in \mathbb{Z}^{k \times k}$: roughly-sorted matrix

```

1 function FOURWAYMERGE(M)
2   for  $i \leftarrow 0$  to  $k - 1$  do
3     if  $i < \frac{k}{2}$  then
4        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"ASC"})$ 
5     else
6        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"DESC"})$ 
7     end if
8   end for
9   for  $j \leftarrow 0$  to  $k - 1$  do
10     $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
11  end for
12  for  $i \leftarrow 0$  to  $k - 1$  do
13    if  $i \bmod 2 = 1$  then
14       $M'[i, :] \leftarrow \text{SORT}(M_{\text{tmp}}[i, :], \text{"ASC"})$ 
15    else
16       $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"DESC"})$ 
17    end if
18  end for
19  for  $j \leftarrow 0$  to  $k - 1$  do
20     $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
21  end for
22 end function

```

After sort the rows of the submatrix (algorithm 18)

$$\begin{bmatrix} 0 & 1 \\ 5 & 4 \end{bmatrix}$$

After sort the columns (algorithm 18)

$$\begin{bmatrix} 0 & 1 \\ 5 & 4 \end{bmatrix}$$

After sort the rows in alternating order (algorithm 18)

$$\begin{bmatrix} 1 & 0 \\ 4 & 5 \end{bmatrix}$$

After sort the columns (algorithm 18)

$$\begin{bmatrix} 1 & 0 \\ 4 & 5 \end{bmatrix}$$

Algorithm 19 Roughsort Algorithm**Input:** $M \in \mathbb{Z}^{k \times k}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{k \times k}$: roughly-sorted matrix

```

1 function ROUGHSORT(M)
2   if  $k > 1$  then
3     subNW  $\leftarrow$  GETSUBMATRIX(M, "NW")
4     subNW'  $\leftarrow$  ROUGHSORT(subNW)
5     subNE  $\leftarrow$  GETSUBMATRIX(M, "NE")
6     subNE'  $\leftarrow$  ROUGHSORT(subNE)
7     subSW  $\leftarrow$  GETSUBMATRIX(M, "SW")
8     subSW'  $\leftarrow$  ROUGHSORT(subSW)
9     subSE  $\leftarrow$  GETSUBMATRIX(M, "SE")
10    subSE'  $\leftarrow$  ROUGHSORT(subSE)
11    M'  $\leftarrow$ 
        ASSEMBLE(subNW', subNE', subSW', subSE')
12    M'  $\leftarrow$  FOURWAYMERGE(M')
13    return M'
14  else
15    return M
16  end if
17 end function

```

Apply merge to the submatrixNE

$$\begin{bmatrix} 7 & 12 \\ 9 & 8 \end{bmatrix}$$

Algorithm 20 4-Way Merge Algorithm**Input:** $M \in \mathbb{Z}^{k \times k}$: unsorted matrix, whose submatrices of size $\frac{k}{2} \times \frac{k}{2}$ are roughly-sorted**Output:** $M' \in \mathbb{Z}^{k \times k}$: roughly-sorted matrix

```

1 function FOURWAYMERGE(M)
2   for  $i \leftarrow 0$  to  $k - 1$  do
3     if  $i < \frac{k}{2}$  then
4        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"ASC"})$ 
5     else
6        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"DESC"})$ 
7     end if
8   end for
9   for  $j \leftarrow 0$  to  $k - 1$  do
10     $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
11  end for
12  for  $i \leftarrow 0$  to  $k - 1$  do
13    if  $i \bmod 2 = 1$  then
14       $M'[i, :] \leftarrow \text{SORT}(M_{\text{tmp}}[i, :], \text{"ASC"})$ 
15    else
16       $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"DESC"})$ 
17    end if
18  end for
19  for  $j \leftarrow 0$  to  $k - 1$  do
20     $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
21  end for
22 end function

```

After sort the rows of the submatrix (algorithm 20)

$$\begin{bmatrix} 7 & 12 \\ 9 & 8 \end{bmatrix}$$

After sort the columns (algorithm 20)

$$\begin{bmatrix} 7 & 8 \\ 9 & 12 \end{bmatrix}$$

After sort the rows in alternating order (algorithm 20)

$$\begin{bmatrix} 8 & 7 \\ 9 & 12 \end{bmatrix}$$

After sort the columns (algorithm 20)

$$\begin{bmatrix} 8 & 7 \\ 9 & 12 \end{bmatrix}$$

Algorithm 21 Roughsort Algorithm

Input: $M \in \mathbb{Z}^{k \times k}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{k \times k}$: roughly-sorted matrix

```

1 function ROUGHSORT(M)
2   if  $k > 1$  then
3     subNW  $\leftarrow$  GETSUBMATRIX(M, "NW")
4     subNW'  $\leftarrow$  ROUGHSORT(subNW)
5     subNE  $\leftarrow$  GETSUBMATRIX(M, "NE")
6     subNE'  $\leftarrow$  ROUGHSORT(subNE)
7     subSW  $\leftarrow$  GETSUBMATRIX(M, "SW")
8     subSW'  $\leftarrow$  ROUGHSORT(subSW)
9     subSE  $\leftarrow$  GETSUBMATRIX(M, "SE")
10    subSE'  $\leftarrow$  ROUGHSORT(subSE)
11    M'  $\leftarrow$ 
        ASSEMBLE(subNW', subNE', subSW', subSE')
12    M'  $\leftarrow$  FOURWAYMERGE(M')
13    return M'
14  else
15    return M
16  end if
17 end function

```

Apply merge to the submatrixSW

$$\begin{bmatrix} 13 & 6 \\ 3 & 2 \end{bmatrix}$$

Algorithm 22 4-Way Merge Algorithm**Input:** $M \in \mathbb{Z}^{k \times k}$: unsorted matrix, whose submatrices of size $\frac{k}{2} \times \frac{k}{2}$ are roughly-sorted**Output:** $M' \in \mathbb{Z}^{k \times k}$: roughly-sorted matrix

```

1 function FOURWAYMERGE(M)
2   for  $i \leftarrow 0$  to  $k - 1$  do
3     if  $i < \frac{k}{2}$  then
4        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"ASC"})$ 
5     else
6        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"DESC"})$ 
7     end if
8   end for
9   for  $j \leftarrow 0$  to  $k - 1$  do
10     $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
11  end for
12  for  $i \leftarrow 0$  to  $k - 1$  do
13    if  $i \bmod 2 = 1$  then
14       $M'[i, :] \leftarrow \text{SORT}(M_{\text{tmp}}[i, :], \text{"ASC"})$ 
15    else
16       $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"DESC"})$ 
17    end if
18  end for
19  for  $j \leftarrow 0$  to  $k - 1$  do
20     $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
21  end for
22 end function

```

After sort the rows of the submatrix (algorithm 22)

$$\begin{bmatrix} 6 & 13 \\ 3 & 2 \end{bmatrix}$$

After sort the columns (algorithm 22)

$$\begin{bmatrix} 3 & 2 \\ 6 & 13 \end{bmatrix}$$

After sort the rows in alternating order (algorithm 22)

$$\begin{bmatrix} 3 & 2 \\ 6 & 13 \end{bmatrix}$$

After sort the columns (algorithm 22)

$$\begin{bmatrix} 3 & 2 \\ 6 & 13 \end{bmatrix}$$

Algorithm 23 Roughsort Algorithm**Input:** $M \in \mathbb{Z}^{k \times k}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{k \times k}$: roughly-sorted matrix

```

1 function ROUGHSORT(M)
2   if  $k > 1$  then
3     subNW  $\leftarrow$  GETSUBMATRIX(M, "NW")
4     subNW'  $\leftarrow$  ROUGHSORT(subNW)
5     subNE  $\leftarrow$  GETSUBMATRIX(M, "NE")
6     subNE'  $\leftarrow$  ROUGHSORT(subNE)
7     subSW  $\leftarrow$  GETSUBMATRIX(M, "SW")
8     subSW'  $\leftarrow$  ROUGHSORT(subSW)
9     subSE  $\leftarrow$  GETSUBMATRIX(M, "SE")
10    subSE'  $\leftarrow$  ROUGHSORT(subSE)
11    M'  $\leftarrow$ 
        ASSEMBLE(subNW', subNE', subSW', subSE')
12    M'  $\leftarrow$  FOURWAYMERGE(M')
13    return M'
14  else
15    return M
16  end if
17 end function

```

Apply merge to the submatrixSE

$$\begin{bmatrix} 15 & 14 \\ 11 & 10 \end{bmatrix}$$

Algorithm 24 4-Way Merge Algorithm**Input:** $M \in \mathbb{Z}^{k \times k}$: unsorted matrix, whose submatrices of size $\frac{k}{2} \times \frac{k}{2}$ are roughly-sorted**Output:** $M' \in \mathbb{Z}^{k \times k}$: roughly-sorted matrix

```

1 function FOURWAYMERGE(M)
2   for  $i \leftarrow 0$  to  $k - 1$  do
3     if  $i < \frac{k}{2}$  then
4        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"ASC"})$ 
5     else
6        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"DESC"})$ 
7     end if
8   end for
9   for  $j \leftarrow 0$  to  $k - 1$  do
10     $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
11  end for
12  for  $i \leftarrow 0$  to  $k - 1$  do
13    if  $i \bmod 2 = 1$  then
14       $M'[i, :] \leftarrow \text{SORT}(M_{\text{tmp}}[i, :], \text{"ASC"})$ 
15    else
16       $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"DESC"})$ 
17    end if
18  end for
19  for  $j \leftarrow 0$  to  $k - 1$  do
20     $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
21  end for
22 end function

```

After sort the rows of the submatrix (algorithm 24)

$$\begin{bmatrix} 14 & 15 \\ 11 & 10 \end{bmatrix}$$

After sort the columns (algorithm 24)

$$\begin{bmatrix} 11 & 10 \\ 14 & 15 \end{bmatrix}$$

After sort the rows in alternating order (algorithm 24)

$$\begin{bmatrix} 11 & 10 \\ 14 & 15 \end{bmatrix}$$

After sort the columns (algorithm 24)

$$\begin{bmatrix} 11 & 10 \\ 14 & 15 \end{bmatrix}$$

Algorithm 25 Roughsort Algorithm**Input:** $M \in \mathbb{Z}^{k \times k}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{k \times k}$: roughly-sorted matrix

```

1 function ROUGHSORT(M)
2   if  $k > 1$  then
3     subNW  $\leftarrow$  GETSUBMATRIX(M, "NW")
4     subNW'  $\leftarrow$  ROUGHSORT(subNW)
5     subNE  $\leftarrow$  GETSUBMATRIX(M, "NE")
6     subNE'  $\leftarrow$  ROUGHSORT(subNE)
7     subSW  $\leftarrow$  GETSUBMATRIX(M, "SW")
8     subSW'  $\leftarrow$  ROUGHSORT(subSW)
9     subSE  $\leftarrow$  GETSUBMATRIX(M, "SE")
10    subSE'  $\leftarrow$  ROUGHSORT(subSE)
11    M'  $\leftarrow$ 
        ASSEMBLE(subNW', subNE', subSW', subSE')
12    M'  $\leftarrow$  FOURWAYMERGE(M')
13    return M'
14  else
15    return M
16  end if
17 end function

```

Apply merge to the matrix, whose submatrices are roughly sorted

$$\begin{bmatrix} 1 & 0 & 8 & 7 \\ 4 & 5 & 9 & 12 \\ 3 & 2 & 11 & 10 \\ 6 & 13 & 14 & 15 \end{bmatrix}$$

Algorithm 26 4-Way Merge Algorithm**Input:** $M \in \mathbb{Z}^{k \times k}$: unsorted matrix, whose submatrices of size $\frac{k}{2} \times \frac{k}{2}$ are roughly-sorted**Output:** $M' \in \mathbb{Z}^{k \times k}$: roughly-sorted matrix

```

1 function FOURWAYMERGE(M)
2   for  $i \leftarrow 0$  to  $k - 1$  do
3     if  $i < \frac{k}{2}$  then
4        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"ASC"})$ 
5     else
6        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"DESC"})$ 
7     end if
8   end for
9   for  $j \leftarrow 0$  to  $k - 1$  do
10     $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
11  end for
12  for  $i \leftarrow 0$  to  $k - 1$  do
13    if  $i \bmod 2 = 1$  then
14       $M'[i, :] \leftarrow \text{SORT}(M_{\text{tmp}}[i, :], \text{"ASC"})$ 
15    else
16       $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"DESC"})$ 
17    end if
18  end for
19  for  $j \leftarrow 0$  to  $k - 1$  do
20     $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
21  end for
22 end function

```

After sort the rows of the submatrix (algorithm 26)

$$\begin{bmatrix} 0 & 1 & 7 & 8 \\ 4 & 5 & 9 & 12 \\ 11 & 10 & 3 & 2 \\ 15 & 14 & 13 & 6 \end{bmatrix}$$

After sort the columns (algorithm 26)

$$\begin{bmatrix} 0 & 1 & 3 & 2 \\ 4 & 5 & 7 & 6 \\ 11 & 10 & 9 & 8 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

After sort the rows in alternating order (algorithm 26)

$$\begin{bmatrix} 3 & 2 & 1 & 0 \\ 4 & 5 & 6 & 7 \\ 11 & 10 & 9 & 8 \\ 12 & 13 & 14 & 15 \end{bmatrix}$$

After sort the columns (algorithm 26)

$$\begin{bmatrix} 3 & 2 & 1 & 0 \\ 4 & 5 & 6 & 7 \\ 11 & 10 & 9 & 8 \\ 12 & 13 & 14 & 15 \end{bmatrix}$$

Algorithm 27 4-Way Mergesort Algorithm

Input: $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: row-major sorted matrix

```
1 function FOURWAYMERGESORT(M)
2    $M' \leftarrow \text{ROUGH SORT}(M)$ 
3   for  $i \leftarrow 0$  to  $n - 1$  do
4      $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
5   end for
6   return  $M'$ 
7 end function
```

After roughsort (algorithm 27)

$$\begin{bmatrix} 3 & 2 & 1 & 0 \\ 4 & 5 & 6 & 7 \\ 11 & 10 & 9 & 8 \\ 12 & 13 & 14 & 15 \end{bmatrix}$$

Algorithm 28 4-Way Mergesort Algorithm

Input: $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: row-major sorted matrix

```
1 function FOURWAYMERGESORT(M)
2    $M' \leftarrow \text{ROUGH SORT}(M)$ 
3   for  $i \leftarrow 0$  to  $n - 1$  do
4      $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
5   end for
6   return  $M'$ 
7 end function
```

After last row sorting (algorithm 28)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \\ 12 & 13 & 14 & 15 \end{bmatrix}$$

1 Introduction

2 LS3-Sort

- Algorithms
- Correctness
- Analysis
- Example

3 4-Way Mergesort

- Algorithms
- Correctness
- Analysis
- Example

4 Rotatesort

- Algorithms
- Correctness
- Analysis
- Example

5 3n-Sort of Schnorr and Shamir

- Algorithm
- Correctness
- Analysis
- Example

6 2D Odd-Even Transposition Sort

- Algorithm
- Correctness
- Analysis
- Example

7 Shearsort

- Algorithm
- Correctness
- Analysis
- Example

8 Conclusion

9 References

The rotatesort [3] is characterized by consisting of a constant number of phases. A phase is a maximal sequence of operations that apply to rows or columns, respectively.

The sorting direction is snake-like.

The $n \times n$ -array is decomposed into

- vertical slices: $n \times \sqrt{n}$ -subarray;
- horizontal slices: $\sqrt{n} \times n$ -subarray;
- blocks: $\sqrt{n} \times \sqrt{n}$ -subarray.

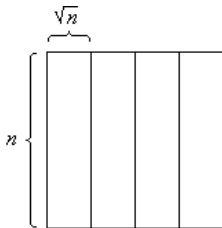


Figure: Vertical Slice

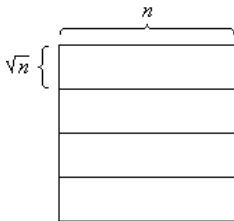


Figure: Horizontal Slice

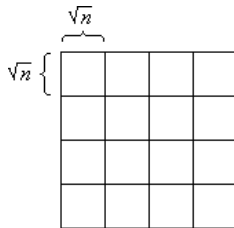


Figure: Block

Definition (Clean)

An $r \times s$ -subarray is **Clean**, if it consists of 0s or 1s only.

- 1 Introduction
- 2 LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 3 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 4 Rotatesort
 - **Algorithms**
 - Correctness
 - Analysis
 - Example
- 5 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 6 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 7 Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 8 Conclusion
- 9 References

Algorithm 29 Rotatesort Algorithm

Input: $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: row-major sorted matrix

```

1 function ROTATESORT(M)
2   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
3      $j \leftarrow k(\sqrt{n} + 1)$ 
4     verticalSlice  $\leftarrow M[:, j \dots j + \sqrt{n}]$ 
5      $M'[:, j \dots j + \sqrt{n}] \leftarrow \text{BALANCE}(\text{verticalSlice})$ 
6   end for
7    $M' \leftarrow \text{UNBLOCK}(M')$ 
8   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do

```

```

9      $i \leftarrow k(\sqrt{n} + 1)$ 
10    horizontalSlice  $\leftarrow M'[i \dots i + \sqrt{n}, :]$ 
11     $M'[i \dots i + \sqrt{n}, :] \leftarrow \text{BALANCE}(\text{horizontalSlice}^\top)^\top$ 
12  end for
13   $M' \leftarrow \text{UNBLOCK}(M')$ 
14   $M' \leftarrow \text{SHEAR}(M')$ 
15   $M' \leftarrow \text{SHEAR}(M')$ 
16   $M' \leftarrow \text{SHEAR}(M')$ 
17  for  $i \leftarrow 0$  to  $n - 1$  do
18     $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
19  end for
20  return  $M'$ 
21 end function

```

Algorithm 30 Balance Operation

Input:

$S \in \mathbb{Z}^{k \times \sqrt{k}}$: unsorted slice

Output:

$S' \in \mathbb{Z}^{k \times \sqrt{k}}$: sorted slice

```
1 function BALANCE(S, text)
2   for  $j \leftarrow 0$  to  $\sqrt{k} - 1$  do
3      $S'[:, j] \leftarrow \text{SORT}(S[:, j], \text{"ASC"})$ 
4   end for
5   for  $i \leftarrow 0$  to  $k - 1$  do
6      $S'[i, :] \leftarrow \text{ROTATE}(S'[i, :], i \bmod \sqrt{k})$ 
7   end for
8   for  $j \leftarrow 0$  to  $\sqrt{k} - 1$  do
9      $S'[:, j] \leftarrow \text{SORT}(S'[:, j], \text{"ASC"})$ 
10  end for
11  return  $S'$ 
12 end function
```

Algorithm 31 Unblock Operation

Input: $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: matrix in which the elements of each block are distributed over the entire width of the matrix

```
1 function UNBLOCK(M)
2   for  $i \leftarrow 0$  to  $n - 1$  do
3      $M'[i, :] \leftarrow \text{ROTATE}(M[i, :], i\sqrt{n} \bmod n)$ 
4   end for
5   for  $j \leftarrow 0$  to  $n - 1$  do
6      $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
7   end for
8   return  $M'$ 
9 end function
```

Algorithm 32 Shear Operation

Input:

$M \in \mathbb{Z}^{n \times n}$: unsorted matrix

Output:

$M' \in \mathbb{Z}^{n \times n}$: matrix with half of dirty rows number

```
1 function SHEAR(M)
2   for  $i \leftarrow 0$  to  $n - 1$  do
3     if  $i \bmod 2 = 0$  then
4        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"ASC"})$ 
5     else
6        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"DESC"})$ 
7     end if
8   end for
9   for  $j \leftarrow 0$  to  $n - 1$  do
10     $M'[:, j] \leftarrow \text{SORT}(M[:, j], \text{"ASC"})$ 
11  end for
12  return  $M'$ 
13 end function
```

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ④ **Rotatesort**
 - Algorithms
 - **Correctness**
 - Analysis
 - Example
- ⑤ 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑧ Conclusion
- ⑨ References

BALANCE *Operation on vertical slice* reduces the number of dirty rows in the slice to at most $\sqrt{n}$. After sorting the columns each column of the slice is distributed to all columns of the slice by the rotation, then sorting again the column the 1's of each column contribute to a certain number of clean 1-rows and, possibly, a dirty row.

Altogether, there remain at most $2\sqrt{n}$ dirty blocks.

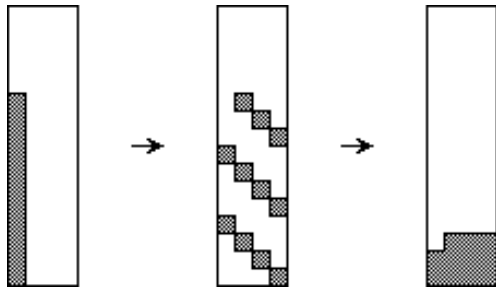


Figure: Balance Operation

UNBLOCK Operation.

- distributes each block to all columns by the rotation;
- sorting the column, so, each clean block generates a clean row and each dirty block generates a dirty row.

Altogether, at most $2\sqrt{n}$ dirty rows that stem from the at most $2\sqrt{n}$ dirty blocks remain.

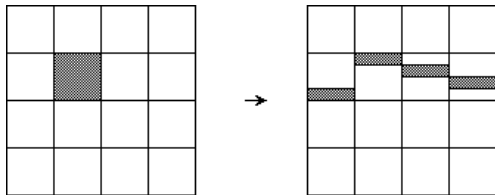


Figure: Unblock Operation

BALANCE Operation on horizontal slice.

The at most $2\sqrt{n}$ dirty rows contribute to at most 3 horizontal slices.

There remain at most 6 dirty blocks.

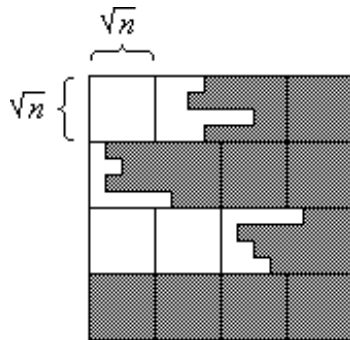


Figure: Balance Operation

UNBLOCK *Operation* distributes the at most 6 dirty blocks to at most 6 dirty rows.

SHEAR Operation reduces every 2 dirty rows to at most 1 dirty row.

- Sorting the rows in alternating order;
- by sorting the columns a clean row is produced from every 2 dirty rows.

By the operation shear the number of dirty rows is halved. By the threefold application of shear the 6 dirty rows are reduced to 1 dirty row.

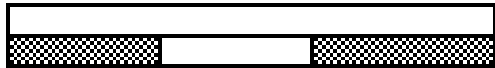
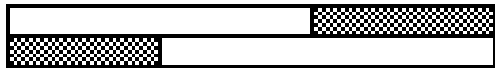


Figure: Shear Operation - clean row of 0

Eventually, *Sort the Last Unsorted Row*

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ④ **Rotatesort**
 - Algorithms
 - Correctness
 - **Analysis**
 - Example
- ⑤ 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑧ Conclusion
- ⑨ References

The analysis of rotatesort yields the following number of phases, where sorting or rotating a row or column:

- BALANCE: 3 phases;
- UNBLOCK: 2 phases;
- BALANCE: 3 phases;
- UNBLOCK: 2 phases;
- SHEAR: 3×2 phase;
- *sorting the last row*: 1 phase.

All phases require at most n elementary steps, so the algorithm is a $\mathcal{O}(17n)$

By a more careful analysis the constant is reduced to 10.

- In the BALANCE Operation, the rotation phases need not be counted, since they require only $\sqrt{n}$ elementary steps;
- in the UNBLOCK Operation, the rotation phase can be realized in $\frac{n}{2}$ elementary steps, by implementing a right rotation of $k > \frac{n}{2}$ and a left rotation of $n - k > \frac{n}{2}$ positions;
- the UNBLOCK Operation produces at most $3\sqrt{n}$ dirty rows; thus a corresponding number of sort steps suffices for sorting the columns.

This implies that the rotatesort requires at most $10n + \mathcal{O}(\sqrt{n})$ elementary steps.

Going to study complexity to constants:

■ BALANCE operation on vertical slices requires:

- ▶ sorting the columns: n steps and $\sqrt{n} \frac{n(n-1)}{2}$ comparators;
- ▶ rotate rows: $\sqrt{n}$ steps and 0 comparators (it is only a permutation);
- ▶ sort columns: n steps and $\sqrt{n} \frac{n(n-1)}{2}$ comparators

■ UNBLOCK operation requires:

- ▶ rotate rows: $\frac{n}{2}$ and 0 comparators;
- ▶ sort columns: n steps and $n \frac{n(n-1)}{2}$ comparators

■ BALANCE operation on horizontal slices requires:

- ▶ sorting the columns: n steps and $\sqrt{n} \frac{n(n-1)}{2}$ comparators;
- ▶ rotate rows: $\sqrt{n}$ steps and 0 comparators (it is only a permutation);
- ▶ sort columns: n steps and $\sqrt{n} \frac{n(n-1)}{2}$ comparators

■ UNBLOCK operation requires:

- ▶ rotate rows: $\frac{n}{2}$ and 0 comparators;
- ▶ sort columns: n steps and $n \frac{n(n-1)}{2}$ comparators

■ three times SHEAR operation requires:

- ▶ sort rows: $3n$ steps and $3n \frac{n(n-1)}{2}$ comparators;
- ▶ sort columns: $6 + 3 + 2$ steps and $n \left(\frac{6(6-1)}{2} + \frac{3(3-1)}{2} + \frac{2(2-1)}{2} \right)$ comparators

- *sort the last unsorted row* requires: n steps and $n \frac{n(n-1)}{2}$ comparators.

So, the final complexity is $T(n^2) = 10n + 5\sqrt{n} + 11$ steps and $C(n^2) = \frac{9}{2}n^3 + \frac{3}{2}n^{\frac{5}{2}} - \frac{9}{2}n^2 - \frac{3}{2}n^{\frac{3}{2}} + 19n$ comparators.

- 1 Introduction
- 2 LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 3 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 4 **Rotatesort**
 - Algorithms
 - Correctness
 - Analysis
 - **Example**
- 5 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 6 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 7 Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 8 Conclusion
- 9 References

Algorithm 33 Rotatesort Algorithm**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: row-major sorted matrix

```

1 function ROTATESORT(M)
2   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
3      $j \leftarrow k(\sqrt{n} + 1)$ 
4     verticalSlice  $\leftarrow M[:, j \dots j + \sqrt{n}]$ 
5      $M'[:, j \dots j + \sqrt{n}] \leftarrow \text{BALANCE}(\text{verticalSlice})$ 
6   end for
7    $M' \leftarrow \text{UNBLOCK}(M')$ 
8   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
9      $i \leftarrow k(\sqrt{n} + 1)$ 
10    horizontalSlice  $\leftarrow M'[i \dots i + \sqrt{n}, :]$ 
11     $M'[i \dots i + \sqrt{n}, :] \leftarrow \text{BALANCE}(\text{horizontalSlice}^\top)^\top$ 
12  end for
13   $M' \leftarrow \text{UNBLOCK}(M')$ 
14   $M' \leftarrow \text{SHEAR}(M')$ 
15   $M' \leftarrow \text{SHEAR}(M')$ 
16   $M' \leftarrow \text{SHEAR}(M')$ 
17  for  $i \leftarrow 0$  to  $n - 1$  do
18     $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
19  end for
20  return  $M'$ 
21 end function

```

Initial situation

$$\begin{bmatrix} 0 & 1 & 7 & 12 \\ 4 & 5 & 9 & 8 \\ 13 & 6 & 15 & 14 \\ 3 & 2 & 11 & 10 \end{bmatrix}$$

Rotatesort Example (2)

Algorithm 34 Balance Operation**Input:** $S \in \mathbb{Z}^{k \times \sqrt{k}}$: unsorted slice**Output:** $S' \in \mathbb{Z}^{k \times \sqrt{k}}$: sorted slice

```

1 function BALANCE(S, text)
2   for  $j \leftarrow 0$  to  $\sqrt{k} - 1$  do
3      $S'[:, j] \leftarrow \text{SORT}(S[:, j], \text{"ASC"})$ 
4   end for
5   for  $i \leftarrow 0$  to  $k - 1$  do
6      $S'[i, :] \leftarrow \text{ROTATE}(S'[i, :], i \bmod \sqrt{k})$ 
7   end for
8   for  $j \leftarrow 0$  to  $\sqrt{k} - 1$  do
9      $S'[:, j] \leftarrow \text{SORT}(S'[:, j], \text{"ASC"})$ 
10  end for
11  return  $S'$ 
12 end function

```

First slice

$$\begin{bmatrix} 0 & 1 \\ 4 & 5 \\ 13 & 6 \\ 3 & 2 \end{bmatrix}$$

After sort columns
(algorithm 34)

$$\begin{bmatrix} 0 & 1 \\ 3 & 2 \\ 4 & 5 \\ 13 & 6 \end{bmatrix}$$

After rotate rows (algorithm 34)

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \\ 4 & 5 \\ 6 & 13 \end{bmatrix}$$

After sort columns
(algorithm 34)

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \\ 4 & 5 \\ 6 & 13 \end{bmatrix}$$

Algorithm 35 Balance Operation**Input:** $S \in \mathbb{Z}^{k \times \sqrt{k}}$: unsorted slice**Output:** $S' \in \mathbb{Z}^{k \times \sqrt{k}}$: sorted slice

```

1 function BALANCE(S, text)
2   for  $j \leftarrow 0$  to  $\sqrt{k} - 1$  do
3      $S'[:, j] \leftarrow \text{SORT}(S[:, j], \text{"ASC"})$ 
4   end for
5   for  $i \leftarrow 0$  to  $k - 1$  do
6      $S'[i, :] \leftarrow \text{ROTATE}(S'[i, :], i \bmod \sqrt{k})$ 
7   end for
8   for  $j \leftarrow 0$  to  $\sqrt{k} - 1$  do
9      $S'[:, j] \leftarrow \text{SORT}(S'[:, j], \text{"ASC"})$ 
10  end for
11  return  $S'$ 
12 end function

```

Second slice

$$\begin{bmatrix} 7 & 12 \\ 9 & 8 \\ 15 & 14 \\ 11 & 10 \end{bmatrix}$$

After sort columns
(algorithm 35)

$$\begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \\ 15 & 14 \end{bmatrix}$$

After rotate rows (algorithm 35)

$$\begin{bmatrix} 7 & 8 \\ 10 & 9 \\ 11 & 12 \\ 14 & 15 \end{bmatrix}$$

After sort columns (algorithm 35)

$$\begin{bmatrix} 7 & 8 \\ 10 & 9 \\ 11 & 12 \\ 14 & 15 \end{bmatrix}$$

Algorithm 36 Rotatesort Algorithm**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: row-major sorted matrix

```

1 function ROTATESORT(M)
2   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
3      $j \leftarrow k(\sqrt{n} + 1)$ 
4     verticalSlice  $\leftarrow M[:, j \dots j + \sqrt{n}]$ 
5      $M'[:, j \dots j + \sqrt{n}] \leftarrow \text{BALANCE}(\text{verticalSlice})$ 
6   end for
7    $M' \leftarrow \text{UNBLOCK}(M')$ 
8   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
9      $i \leftarrow k(\sqrt{n} + 1)$ 
10    horizontalSlice  $\leftarrow M'[i \dots i + \sqrt{n}, :]$ 
11     $M'[i \dots i + \sqrt{n}, :] \leftarrow \text{BALANCE}(\text{horizontalSlice}^\top)^\top$ 
12  end for
13   $M' \leftarrow \text{UNBLOCK}(M')$ 
14   $M' \leftarrow \text{SHEAR}(M')$ 
15   $M' \leftarrow \text{SHEAR}(M')$ 
16   $M' \leftarrow \text{SHEAR}(M')$ 
17  for  $i \leftarrow 0$  to  $n - 1$  do
18     $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
19  end for
20  return  $M'$ 
21 end function

```

After balance vertical slice (algorithm 36)

$$\begin{bmatrix} 0 & 1 & 7 & 8 \\ 2 & 3 & 10 & 9 \\ 4 & 5 & 11 & 12 \\ 6 & 13 & 14 & 15 \end{bmatrix}$$

Algorithm 37 Unblock Operation**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: matrix in which the elements of each block are distributed over the entire width of the matrix

```

1 function UNBLOCK(M)
2   for  $i \leftarrow 0$  to  $n - 1$  do
3      $M'[i, :] \leftarrow \text{ROTATE}(M[i, :], i\sqrt{n} \bmod n)$ 
4   end for
5   for  $j \leftarrow 0$  to  $n - 1$  do
6      $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
7   end for
8   return  $M'$ 
9 end function

```

After rotate rows (algorithm 37)

$$\begin{bmatrix} 0 & 1 & 7 & 8 \\ 10 & 9 & 2 & 3 \\ 4 & 5 & 11 & 12 \\ 14 & 15 & 6 & 13 \end{bmatrix}$$

After sort columns (algorithm 37)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 8 \\ 10 & 9 & 7 & 12 \\ 14 & 15 & 11 & 13 \end{bmatrix}$$

Algorithm 38 Rotatesort Algorithm**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: row-major sorted matrix

```

1 function ROTATESORT(M)
2   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
3      $j \leftarrow k(\sqrt{n} + 1)$ 
4     verticalSlice  $\leftarrow M[:, j \dots j + \sqrt{n}]$ 
5      $M'[:, j \dots j + \sqrt{n}] \leftarrow \text{BALANCE}(\text{verticalSlice})$ 
6   end for
7    $M' \leftarrow \text{UNBLOCK}(M')$ 
8   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
9      $i \leftarrow k(\sqrt{n} + 1)$ 
10    horizontalSlice  $\leftarrow M'[i \dots i + \sqrt{n}, :]$ 
11     $M'[i \dots i + \sqrt{n}, :] \leftarrow \text{BALANCE}(\text{horizontalSlice}^\top)^\top$ 
12  end for
13   $M' \leftarrow \text{UNBLOCK}(M')$ 
14   $M' \leftarrow \text{SHEAR}(M')$ 
15   $M' \leftarrow \text{SHEAR}(M')$ 
16   $M' \leftarrow \text{SHEAR}(M')$ 
17  for  $i \leftarrow 0$  to  $n - 1$  do
18     $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
19  end for
20  return  $M'$ 
21 end function

```

After unblock (algorithm 38)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 8 \\ 10 & 9 & 7 & 12 \\ 14 & 15 & 11 & 13 \end{bmatrix}$$

Rotatesort Example (7)

Algorithm 39 Balance Operation**Input:** $S \in \mathbb{Z}^{k \times \sqrt{k}}$: unsorted slice**Output:** $S' \in \mathbb{Z}^{k \times \sqrt{k}}$: sorted slice

```

1 function BALANCE(S, text)
2   for  $j \leftarrow 0$  to  $\sqrt{k} - 1$  do
3      $S'[:, j] \leftarrow \text{SORT}(S[:, j], \text{"ASC"})$ 
4   end for
5   for  $i \leftarrow 0$  to  $k - 1$  do
6      $S'[i, :] \leftarrow \text{ROTATE}(S'[i, :], i \bmod \sqrt{k})$ 
7   end for
8   for  $j \leftarrow 0$  to  $\sqrt{k} - 1$  do
9      $S'[:, j] \leftarrow \text{SORT}(S'[:, j], \text{"ASC"})$ 
10  end for
11  return  $S'$ 
12 end function

```

The horizontal slice is treated as vertical slice and then transposed.

First slice

$$\begin{bmatrix} 0 & 4 \\ 1 & 5 \\ 2 & 6 \\ 3 & 8 \end{bmatrix}$$

After sort columns (algorithm 39)

$$\begin{bmatrix} 0 & 4 \\ 1 & 5 \\ 2 & 6 \\ 3 & 8 \end{bmatrix}$$

After rotate rows (algorithm 39)

$$\begin{bmatrix} 0 & 4 \\ 5 & 1 \\ 2 & 6 \\ 8 & 3 \end{bmatrix}$$

After sort columns (algorithm 39)

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \\ 5 & 4 \\ 8 & 6 \end{bmatrix}$$

Rotatesort Example (8)

Algorithm 40 Balance Operation**Input:** $S \in \mathbb{Z}^{k \times \sqrt{k}}$: unsorted slice**Output:** $S' \in \mathbb{Z}^{k \times \sqrt{k}}$: sorted slice

```

1 function BALANCE(S, text)
2   for  $j \leftarrow 0$  to  $\sqrt{k} - 1$  do
3      $S'[:, j] \leftarrow \text{SORT}(S[:, j], \text{"ASC"})$ 
4   end for
5   for  $i \leftarrow 0$  to  $k - 1$  do
6      $S'[i, :] \leftarrow \text{ROTATE}(S'[i, :], i \bmod \sqrt{k})$ 
7   end for
8   for  $j \leftarrow 0$  to  $\sqrt{k} - 1$  do
9      $S'[:, j] \leftarrow \text{SORT}(S'[:, j], \text{"ASC"})$ 
10  end for
11  return  $S'$ 
12 end function

```

The horizontal slice is treated as vertical slice and then transposed.

Second slice

$$\begin{bmatrix} 10 & 14 \\ 9 & 15 \\ 7 & 11 \\ 12 & 13 \end{bmatrix}$$

After sort columns (algorithm 40)

$$\begin{bmatrix} 7 & 11 \\ 9 & 13 \\ 10 & 14 \\ 12 & 15 \end{bmatrix}$$

After rotate rows (algorithm 40)

$$\begin{bmatrix} 7 & 11 \\ 13 & 9 \\ 10 & 14 \\ 15 & 12 \end{bmatrix}$$

After sort columns (algorithm 40)

$$\begin{bmatrix} 7 & 9 \\ 10 & 11 \\ 13 & 12 \\ 15 & 14 \end{bmatrix}$$

Algorithm 41 Rotatesort Algorithm**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: row-major sorted matrix

```

1 function ROTATESORT(M)
2   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
3      $j \leftarrow k(\sqrt{n} + 1)$ 
4     verticalSlice  $\leftarrow M[:, j \dots j + \sqrt{n}]$ 
5      $M'[:, j \dots j + \sqrt{n}] \leftarrow \text{BALANCE}(\text{verticalSlice})$ 
6   end for
7    $M' \leftarrow \text{UNBLOCK}(M')$ 
8   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
9      $i \leftarrow k(\sqrt{n} + 1)$ 
10    horizontalSlice  $\leftarrow M'[i \dots i + \sqrt{n}, :]$ 
11     $M'[i \dots i + \sqrt{n}, :] \leftarrow \text{BALANCE}(\text{horizontalSlice}^\top)^\top$ 
12  end for
13   $M' \leftarrow \text{UNBLOCK}(M')$ 
14   $M' \leftarrow \text{SHEAR}(M')$ 
15   $M' \leftarrow \text{SHEAR}(M')$ 
16   $M' \leftarrow \text{SHEAR}(M')$ 
17  for  $i \leftarrow 0$  to  $n - 1$  do
18     $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
19  end for
20  return  $M'$ 
21 end function

```

After balance horizontal slice (algorithm 41)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 5 & 4 & 7 & 6 \\ 10 & 9 & 8 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

Algorithm 42 Unblock Operation**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: matrix in which the elements of each block are distributed over the entire width of the matrix

```

1 function UNBLOCK(M)
2   for  $i \leftarrow 0$  to  $n - 1$  do
3      $M'[i, :] \leftarrow \text{ROTATE}(M[i, :], i\sqrt{n} \bmod n)$ 
4   end for
5   for  $j \leftarrow 0$  to  $n - 1$  do
6      $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
7   end for
8   return  $M'$ 
9 end function

```

After rotate rows (algorithm 42)

$$\begin{bmatrix} 0 & 2 & 5 & 8 \\ 4 & 6 & 1 & 3 \\ 7 & 10 & 13 & 15 \\ 12 & 14 & 9 & 11 \end{bmatrix}$$

After sort columns (algorithm 42)

$$\begin{bmatrix} 0 & 2 & 1 & 3 \\ 4 & 6 & 5 & 8 \\ 7 & 10 & 9 & 11 \\ 12 & 14 & 13 & 15 \end{bmatrix}$$

Algorithm 43 Rotatesort Algorithm**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: row-major sorted matrix

```

1 function ROTATESORT(M)
2   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
3      $j \leftarrow k(\sqrt{n} + 1)$ 
4     verticalSlice  $\leftarrow M[:, j \dots j + \sqrt{n}]$ 
5      $M'[:, j \dots j + \sqrt{n}] \leftarrow \text{BALANCE}(\text{verticalSlice})$ 
6   end for
7    $M' \leftarrow \text{UNBLOCK}(M')$ 
8   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
9      $i \leftarrow k(\sqrt{n} + 1)$ 
10    horizontalSlice  $\leftarrow M'[i \dots i + \sqrt{n}, :]$ 
11     $M'[i \dots i + \sqrt{n}, :] \leftarrow \text{BALANCE}(\text{horizontalSlice}^\top)^\top$ 
12  end for
13   $M' \leftarrow \text{UNBLOCK}(M')$ 
14   $M' \leftarrow \text{SHEAR}(M')$ 
15   $M' \leftarrow \text{SHEAR}(M')$ 
16   $M' \leftarrow \text{SHEAR}(M')$ 
17  for  $i \leftarrow 0$  to  $n - 1$  do
18     $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
19  end for
20  return  $M'$ 
21 end function

```

After unblock (algorithm 43)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 7 & 6 & 5 & 4 \\ 10 & 9 & 8 & 11 \\ 13 & 12 & 15 & 14 \end{bmatrix}$$

Algorithm 44 Shear Operation**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: matrix with half of dirty rows number

```

1 function SHEAR(M)
2   for  $i \leftarrow 0$  to  $n - 1$  do
3     if  $i \bmod 2 = 0$  then
4        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"ASC"})$ 
5     else
6        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"DESC"})$ 
7     end if
8   end for
9   for  $j \leftarrow 0$  to  $n - 1$  do
10     $M'[:, j] \leftarrow \text{SORT}(M[:, j], \text{"ASC"})$ 
11  end for
12  return  $M'$ 
13 end function

```

After sort rows (algorithm 44)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 8 & 6 & 5 & 4 \\ 7 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

After sort columns (algorithm 44)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 7 & 6 & 5 & 4 \\ 8 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

Algorithm 45 Rotatesort Algorithm**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: row-major sorted matrix

```

1 function ROTATESORT(M)
2   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
3      $j \leftarrow k(\sqrt{n} + 1)$ 
4     verticalSlice  $\leftarrow M[:, j \dots j + \sqrt{n}]$ 
5      $M'[:, j \dots j + \sqrt{n}] \leftarrow \text{BALANCE}(\text{verticalSlice})$ 
6   end for
7    $M' \leftarrow \text{UNBLOCK}(M')$ 
8   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
9      $i \leftarrow k(\sqrt{n} + 1)$ 
10    horizontalSlice  $\leftarrow M'[i \dots i + \sqrt{n}, :]$ 
11     $M'[i \dots i + \sqrt{n}, :] \leftarrow \text{BALANCE}(\text{horizontalSlice}^\top)^\top$ 
12  end for
13   $M' \leftarrow \text{UNBLOCK}(M')$ 
14   $M' \leftarrow \text{SHEAR}(M')$ 
15   $M' \leftarrow \text{SHEAR}(M')$ 
16   $M' \leftarrow \text{SHEAR}(M')$ 
17  for  $i \leftarrow 0$  to  $n - 1$  do
18     $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
19  end for
20  return  $M'$ 
21 end function

```

After first shear (algorithm 45)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 7 & 6 & 5 & 4 \\ 8 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

Algorithm 46 Shear Operation**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: matrix with half of dirty rows number

```

1 function SHEAR(M)
2   for  $i \leftarrow 0$  to  $n - 1$  do
3     if  $i \bmod 2 = 0$  then
4        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"ASC"})$ 
5     else
6        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"DESC"})$ 
7     end if
8   end for
9   for  $j \leftarrow 0$  to  $n - 1$  do
10     $M'[:, j] \leftarrow \text{SORT}(M[:, j], \text{"ASC"})$ 
11  end for
12  return  $M'$ 
13 end function

```

After sort rows (algorithm 46)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 8 & 6 & 5 & 4 \\ 7 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

After sort columns (algorithm 46)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 7 & 6 & 5 & 4 \\ 8 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

Algorithm 47 Rotatesort Algorithm**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: row-major sorted matrix

```

1 function ROTATESORT(M)
2   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
3      $j \leftarrow k(\sqrt{n} + 1)$ 
4     verticalSlice  $\leftarrow M[:, j \dots j + \sqrt{n}]$ 
5      $M'[:, j \dots j + \sqrt{n}] \leftarrow \text{BALANCE}(\text{verticalSlice})$ 
6   end for
7    $M' \leftarrow \text{UNBLOCK}(M')$ 
8   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
9      $i \leftarrow k(\sqrt{n} + 1)$ 
10    horizontalSlice  $\leftarrow M'[i \dots i + \sqrt{n}, :]$ 
11     $M'[i \dots i + \sqrt{n}, :] \leftarrow \text{BALANCE}(\text{horizontalSlice}^\top)^\top$ 
12  end for
13   $M' \leftarrow \text{UNBLOCK}(M')$ 
14   $M' \leftarrow \text{SHEAR}(M')$ 
15   $M' \leftarrow \text{SHEAR}(M')$ 
16   $M' \leftarrow \text{SHEAR}(M')$ 
17  for  $i \leftarrow 0$  to  $n - 1$  do
18     $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
19  end for
20  return  $M'$ 
21 end function

```

After second shear (algorithm 47)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 7 & 6 & 5 & 4 \\ 8 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

Algorithm 48 Shear Operation**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: matrix with half of dirty rows number

```

1 function SHEAR(M)
2   for  $i \leftarrow 0$  to  $n - 1$  do
3     if  $i \bmod 2 = 0$  then
4        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"ASC"})$ 
5     else
6        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"DESC"})$ 
7     end if
8   end for
9   for  $j \leftarrow 0$  to  $n - 1$  do
10     $M'[:, j] \leftarrow \text{SORT}(M[:, j], \text{"ASC"})$ 
11  end for
12  return  $M'$ 
13 end function

```

After sort rows (algorithm 48)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 8 & 6 & 5 & 4 \\ 7 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

After sort columns (algorithm 48)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 7 & 6 & 5 & 4 \\ 8 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

Algorithm 49 Rotatesort Algorithm**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: row-major sorted matrix

```

1 function ROTATESORT(M)
2   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
3      $j \leftarrow k(\sqrt{n} + 1)$ 
4     verticalSlice  $\leftarrow M[:, j \dots j + \sqrt{n}]$ 
5      $M'[:, j \dots j + \sqrt{n}] \leftarrow \text{BALANCE}(\text{verticalSlice})$ 
6   end for
7    $M' \leftarrow \text{UNBLOCK}(M')$ 
8   for  $k \leftarrow 0$  to  $\sqrt{n} - 1$  do
9      $i \leftarrow k(\sqrt{n} + 1)$ 
10    horizontalSlice  $\leftarrow M'[i \dots i + \sqrt{n}, :]$ 
11     $M'[i \dots i + \sqrt{n}, :] \leftarrow \text{BALANCE}(\text{horizontalSlice}^\top)^\top$ 
12  end for
13   $M' \leftarrow \text{UNBLOCK}(M')$ 
14   $M' \leftarrow \text{SHEAR}(M')$ 
15   $M' \leftarrow \text{SHEAR}(M')$ 
16   $M' \leftarrow \text{SHEAR}(M')$ 
17  for  $i \leftarrow 0$  to  $n - 1$  do
18     $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
19  end for
20  return  $M'$ 
21 end function

```

After third shear (algorithm 49)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 7 & 6 & 5 & 4 \\ 8 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

After sorting the last row (algorithm 49)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 7 & 6 & 5 & 4 \\ 8 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ **3n-Sort of Schnorr and Shamir**
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑧ Conclusion
- ⑨ References

The algorithm 3n-sort of Schnorr and Shamir [6] is more complicated than rotatesort [3] and 4-way mergesort [5].

The sorting direction is snake-like.

The $n \times n$ -array is decomposed into

- vertical slices: $n \times n^{\frac{3}{4}}$ -subarray;
- horizontal slices: $n^{\frac{3}{4}} \times n$ -subarray;
- blocks: $n^{\frac{3}{4}} \times n^{\frac{3}{4}}$ -subarray.

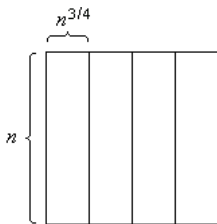


Figure: Vertical Slice

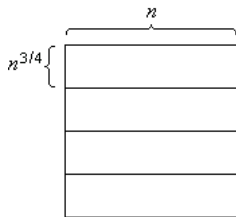


Figure: Horizontal Slice

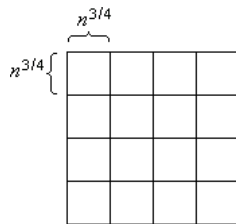


Figure: Block

Definition (k-way Unshuffle)

A **k-way Unshuffle** is a permutation that corresponds to dealing n cards to k players, it requires the $k \mid n$.

Example (4-Way Unshuffle)

Suppose to have 12 numbers, $0, 1, \dots, 11$ and 4 players:

- player 1 receives: 0, 4, 8;
- player 2 receives: 1, 5, 9;
- player 3 receives: 2, 6, 10;
- player 4 receives: 3, 7, 11.

In a mathematical way:

- set size: n , in the example $n = 12$;
- number of way: k , in the example $k = 4$;
- block size: $m = \lceil \frac{n}{k} \rceil$, in the example $m = 3$;
- k-way unshuffle permutation:

$$\sigma : \{0, \dots, n-1\} \rightarrow \{0, \dots, n-1\}$$

$$\sigma : i \mapsto (i \bmod k) \cdot m + \left\lfloor \frac{i}{k} \right\rfloor$$

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ **3n-Sort of Schnorr and Shamir**
 - **Algorithm**
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑧ Conclusion
- ⑨ References

Algorithm 50 3n-Sort Algorithm**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```

1 function THREESORT(M)
2   for  $i \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
3     for  $j \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
4       block  $\leftarrow M[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1]$ 
5        $M'[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1] \leftarrow$ 
        SORT(block, "ASC")
6     end for
7   end for
8   for  $i \leftarrow 0$  to  $n - 1$  do
9      $M' \leftarrow \text{kWayUNSHUFFLE}(M[i, :], n^{1/4})$ 
10  end for
11  for  $i \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
12    for  $j \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
13      block  $\leftarrow M[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1]$ 
14       $M'[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1] \leftarrow$ 
        SORT(block, "ASC")

```

```

15   end for
16 end for
17 for  $j \leftarrow 0$  to  $n - 1$  do
18    $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
19 end for
20 for  $k \leftarrow 0$  to  $n^{1/4} - 1$  do
21    $j \leftarrow k \left( n^{1/4} + 1 \right)$ 
22   verticalSlice  $\leftarrow M'[:, j \dots j + n^{1/4}]$ 
23    $M'[:, j \dots j + n^{1/4}] \leftarrow \text{SORT}(\text{verticalSlice}, \text{"ASC"})$ 
24 end for
25 for  $i \leftarrow 0$  to  $n - 1$  do
26   if  $i \bmod 2 = 0$  then
27      $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
28   else
29      $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"DESC"})$ 
30   end if
31 end for
32  $M' \leftarrow \text{oETS}(M')$ 
33 return  $M'$ 
34 end function

```

- 1 Introduction
- 2 LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 3 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 4 Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 5 **3n-Sort of Schnorr and Shamir**
 - Algorithm
 - **Correctness**
 - Analysis
 - Example
- 6 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 7 Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 8 Conclusion
- 9 References

Sort the blocks:

after sorting the blocks each block has at most one incomplete row.

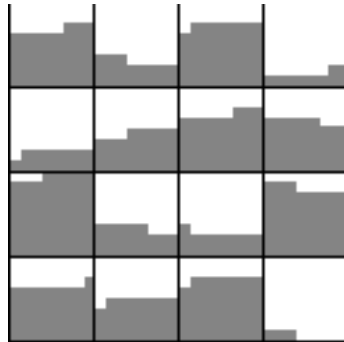


Figure: Sort the Block

$n^{\frac{1}{4}}$ -way unshuffle along the rows distributes the columns of a block to all $n^{\frac{1}{4}}$ blocks of the same horizontal slice in a round-robin way.

If a block has an incomplete row, some blocks receive a one more from this block than others

Altogether, a block can receive at most $n^{\frac{1}{4}}$ ones more than any other.

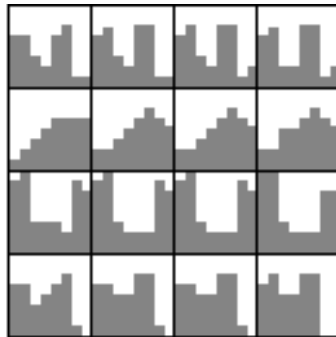


Figure: $n^{\frac{1}{4}}$ -way unshuffle

Sort the block:

after sorting the blocks again each block contains at most one incomplete row.

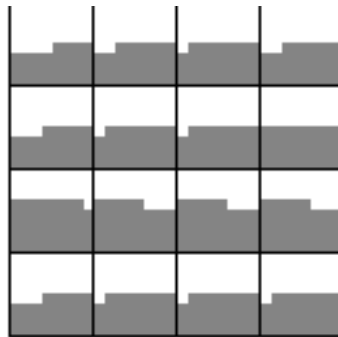


Figure: Sort the Block

Sort the Columns:

after sorting the columns each vertical slice has at most $n^{\frac{1}{4}}$ dirty rows.

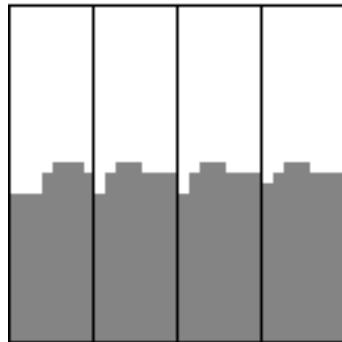


Figure: Sort the Columns

Sort the Vertical Slices:

after sorting the vertical slices all vertical slices
have almost the same number of complete
1-rows: the difference can be at most 1.

The region of length $n^{\frac{1}{2}}$ on the snake in each
vertical slice that can contain these additional 1s.

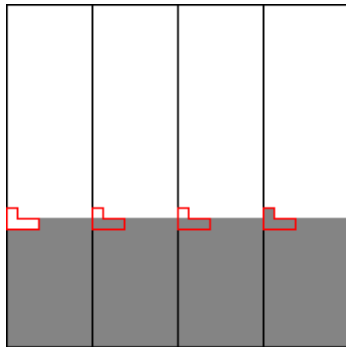


Figure: Sort the Vertical Slices

Sort the Rows in Alternating Order:
the two last dirty rows are sorted in alternating direction.

Possibly still unsorted are the at most
 $n^{\frac{1}{4}} n^{\frac{1}{2}} = n^{\frac{3}{4}}$.

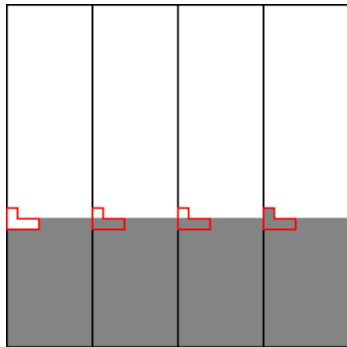


Figure: Sort the Rows

Apply $n^{\frac{3}{4}}$ -steps of OETS to the Snake:
the unsorted zone of length at most $n^{\frac{3}{4}}$ is sorted according to the snake.

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ **3n-Sort of Schnorr and Shamir**
 - Algorithm
 - Correctness
 - **Analysis**
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑧ Conclusion
- ⑨ References

- *sorting the block*: $\mathcal{O}\left(n^{\frac{3}{4}}\right)$;
- `KWAYUNSHUFFLE`: n ;
- *sorting the block*: $\mathcal{O}\left(n^{\frac{3}{4}}\right)$;
- *sorting the columns*: n ;
- *sorting the vertical slices*: $\mathcal{O}\left(n^{\frac{3}{4}}\right)$;
- *sorting the rows*: n ;
- OETS steps: $\mathcal{O}\left(n^{\frac{3}{4}}\right)$.

This implies that the 3n-sort of Schnorr and Shamir requires at most $3n + \mathcal{O}\left(n^{\frac{3}{4}}\right)$.

3n-Sort Analysis (2)

Going to study complexity to constants:

- *sorting the block*: can be sorted using any linear sorting algorithm for example the 4-way mergesort [5], this implies that $7n^{\frac{3}{4}} - 6$ steps and

$$n^{\frac{1}{2}} \left(4 \left(n^{\frac{3}{4}} \right)^3 - 2 \left(n^{\frac{3}{4}} \right)^2 \log_2 n^{\frac{3}{4}} - \frac{7}{2} \left(n^{\frac{3}{4}} \right)^2 \right)$$

comparators are required;

- **KWAYUNSHUFFLE**: n steps and no comparators, as it is simply a permutation;
- *sorting the block*: as before;
- *sorting the vertical slices*: for example, by sorting the blocks and subsequently the vertically overlapping blocks of $n^{\frac{1}{4}}$ rows. This requires

$7n^{\frac{3}{4}} - 6$ steps and

$$n^{\frac{1}{2}} \left(4 \left(n^{\frac{3}{4}} \right)^3 - 2 \left(n^{\frac{3}{4}} \right)^2 \log_2 n^{\frac{3}{4}} - \frac{7}{2} \left(n^{\frac{3}{4}} \right)^2 \right)$$

comparators;

- *sorting the rows*: n steps and $n^{\frac{n(n-1)}{2}}$ comparators;
- **OETS** steps [1]: each step is a comparison-exchange phase, so requires $n^{\frac{3}{4}}$ steps and $n^{\frac{3}{4}} \frac{n^2}{2}$ comparators.

So, the final complexity is $T(n^2) = 3n + 22n^{\frac{3}{4}} - 18$ steps and $C(n^2) = n^3 + \frac{25}{2}n^{\frac{11}{4}} - \frac{9}{2}n^2 \log_2 n - 13n^2$ comparators.

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ **3n-Sort of Schnorr and Shamir**
 - Algorithm
 - Correctness
 - Analysis
 - **Example**
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑧ Conclusion
- ⑨ References

Algorithm 51 3n-Sort Algorithm**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```

1 function THREE_SORT(M)
2   for  $i \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
3     for  $j \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
4       block  $\leftarrow M[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1]$ 
5        $M'[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1] \leftarrow \text{SORT}(\text{block}, \text{"ASC"})$ 
6     end for
7   end for
8   for  $i \leftarrow 0$  to  $n - 1$  do
9      $M' \leftarrow \text{kWayUnshuffle}(M[i, :], n^{1/4})$ 
10  end for
11  for  $i \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
12    for  $j \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
13      block  $\leftarrow M[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1]$ 
14       $M'[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1] \leftarrow \text{SORT}(\text{block}, \text{"ASC"})$ 
15    end for
16  end for
17  for  $j \leftarrow 0$  to  $n - 1$  do
18     $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
19  end for
20  for  $k \leftarrow 0$  to  $n^{1/4} - 1$  do
21     $j \leftarrow k \left( n^{1/4} + 1 \right)$ 
22    verticalSlice  $\leftarrow M'[:, j \dots j + n^{1/4}]$ 
23     $M'[:, j \dots j + n^{1/4}] \leftarrow \text{SORT}(\text{verticalSlice}, \text{"ASC"})$ 
24  end for
25  for  $i \leftarrow 0$  to  $n - 1$  do
26    if  $i \bmod 2 = 0$  then
27       $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
28    else
29       $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"DESC"})$ 
30    end if
31  end for
32   $M' \leftarrow \text{OETS}(M')$ 
33  return  $M'$ 
34 end function

```

Initial situation

$$\begin{bmatrix} 0 & 1 & 7 & 12 \\ 4 & 5 & 9 & 8 \\ 13 & 6 & 15 & 14 \\ 3 & 2 & 11 & 10 \end{bmatrix}$$

3n-Sort of Schnorr and Shamir - Example

3n-Sort Example (2)

127/174

Algorithm 52 3n-Sort Algorithm

Input: $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```
1 function THREE_SORT(M)
2   for  $i \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
3     for  $j \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
4       block  $\leftarrow M[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1]$ 
5        $M'[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1] \leftarrow \text{SORT}(\text{block}, \text{"ASC"})$ 
6     end for
7   end for
8   for  $i \leftarrow 0$  to  $n - 1$  do
9      $M' \leftarrow \text{kWAYUNSHUFFLE}(M[i, :], n^{1/4})$ 
10  end for
11  for  $i \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
12    for  $j \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
13      block  $\leftarrow M[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1]$ 
14       $M'[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1] \leftarrow \text{SORT}(\text{block}, \text{"ASC"})$ 
15    end for
16  end for
17  for  $j \leftarrow 0$  to  $n - 1$  do
18     $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
19  end for
20  for  $k \leftarrow 0$  to  $\frac{n^{1/4}}{2} - 1$  do
21     $j \leftarrow k \left( \frac{n^{1/4}}{2} + 1 \right)$ 
22    verticalSlice  $\leftarrow M'[:, j \dots j + n^{1/4}]$ 
23     $M'[:, j \dots j + n^{1/4}] \leftarrow \text{SORT}(\text{verticalSlice}, \text{"ASC"})$ 
24  end for
25  for  $i \leftarrow 0$  to  $n - 1$  do
26    if  $i \bmod 2 = 0$  then
27       $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
28    else
29       $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"DESC"})$ 
30    end if
31  end for
32   $M' \leftarrow \text{OETS}(M')$ 
33  return  $M'$ 
34 end function
```

After sort the blocks
(algorithm 52)

$$\begin{bmatrix} 0 & 1 & 7 & 8 \\ 4 & 5 & 9 & 12 \\ 2 & 3 & 10 & 11 \\ 6 & 13 & 14 & 15 \end{bmatrix}$$

3n-Sort of Schnorr and Shamir - Example

3n-Sort Example (3)

128/174

Algorithm 53 3n-Sort Algorithm

```
Input:
  M ∈ ℤn×n: unsorted matrix
Output:
  M' ∈ ℤn×n: snake-sorted matrix

1 function THREESORT(M)
2   for i ← 0 to n - 1 step n3/4 do
3     for j ← 0 to n - 1 step n3/4 do
4       block ← M[i...i + n3/4 -
5         1, j...j + n3/4 - 1, j...j +
6         n3/4 - 1] ← SORT(block, "ASC")
7     end for
8   for i ← 0 to n - 1 do
9     M' ← KWAYUNSHUFFLE(M[i, :], n1/4)
10  end for
11  for i ← 0 to n - 1 step n3/4 do
12    for j ← 0 to n - 1 step n3/4 do
13      block ← M[i...i + n3/4 -
14        1, j...j + n3/4 - 1, j...j +
15        n3/4 - 1] ← SORT(block, "ASC")
16    end for
17    for j ← 0 to n - 1 do
18      M'[:, j] ← SORT(M'[:, j], "ASC")
19    end for
20    for k ← 0 to n1/4 - 1 do
21      j ← k (n1/4 + 1)
22      verticalSlice ← M'[:, j...j + n1/4]
23      M'[:, j...j + n1/4] ←
24        SORT(verticalSlice, "ASC")
25    end for
26    for i ← 0 to n - 1 do
27      if i mod 2 = 0 then
28        M'[i, :] ← SORT(M'[i, :], "ASC")
29      else
30        M'[i, :] ← SORT(M'[i, :], "DESC")
31      end if
32    end for
33    M' ← OETS(M')
34  end function
```

After apply $n^{\frac{3}{4}}$ -way
unshuffle along the rows
(algorithm 53)

$$\begin{bmatrix} 0 & 7 & 1 & 8 \\ 4 & 9 & 5 & 12 \\ 2 & 10 & 3 & 11 \\ 6 & 14 & 13 & 15 \end{bmatrix}$$

3n-Sort of Schnorr and Shamir - Example

3n-Sort Example (4)

129/174

Algorithm 54 3n-Sort Algorithm

```
Input:
  M ∈ ℤn×n: unsorted matrix
Output:
  M' ∈ ℤn×n: snake-sorted matrix

1 function THREESORT(M)
2   for i ← 0 to n - 1 step n3/4 do
3     for j ← 0 to n - 1 step n3/4 do
4       block ← M[i...i + n3/4 -
5         1, j...j + n3/4 - 1, j...j +
6         n3/4 - 1] ← SORT(block, "ASC")
7     end for
8   end for
9   M' ← KWAYUNSHUFFLE(M[i, :], n1/4)
10  end for
11  for i ← 0 to n - 1 step n3/4 do
12    for j ← 0 to n - 1 step n3/4 do
13      block ← M[i...i + n3/4 -
14        1, j...j + n3/4 - 1, j...j +
15        n3/4 - 1] ← SORT(block, "ASC")
16    end for
17    for j ← 0 to n - 1 do
18      M'[:, j] ← SORT(M'[:, j], "ASC")
19    end for
20    for k ← 0 to n1/4 - 1 do
21      j ← k (n1/4 + 1)
22      verticalSlice ← M'[:, j...j + n1/4]
23      M'[:, j...j + n1/4] ←
24        SORT(verticalSlice, "ASC")
25    end for
26    for i ← 0 to n - 1 do
27      if i mod 2 = 0 then
28        M'[i, :] ← SORT(M'[i, :], "ASC")
29      else
30        M'[i, :] ← SORT(M'[i, :], "DESC")
31      end if
32    end for
33    M' ← OETS(M')
34  return M'
end function
```

After sort the blocks
(algorithm 54)

$$\begin{bmatrix} 0 & 4 & 1 & 5 \\ 7 & 9 & 8 & 12 \\ 2 & 6 & 3 & 11 \\ 10 & 14 & 13 & 15 \end{bmatrix}$$

3n-Sort of Schnorr and Shamir - Example

3n-Sort Example (5)

130/174

Algorithm 55 3n-Sort Algorithm

Input: $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```
1 function THREE_SORT(M)
2   for  $i \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
3     for  $j \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
4       block  $\leftarrow M[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1] \leftarrow \text{SORT}(\text{block}, \text{"ASC"})$ 
5        $M'[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1] \leftarrow \text{SORT}(\text{block}, \text{"ASC"})$ 
6     end for
7   end for
8   for  $i \leftarrow 0$  to  $n - 1$  do
9      $M' \leftarrow \text{kWAYUNSHUFFLE}(M[i, :], n^{1/4})$ 
10  end for
11  for  $i \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
12    for  $j \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
13      block  $\leftarrow M[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1]$ 
14       $M'[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1] \leftarrow \text{SORT}(\text{block}, \text{"ASC"})$ 
15    end for
16  end for
17  for  $j \leftarrow 0$  to  $n - 1$  do
18     $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
19  end for
20  for  $k \leftarrow 0$  to  $\frac{n^{1/4}}{2} - 1$  do
21     $j \leftarrow k \left( \frac{n^{1/4}}{2} + 1 \right)$ 
22    verticalSlice  $\leftarrow M'[:, j \dots j + n^{1/4}]$ 
23     $M'[:, j \dots j + n^{1/4}] \leftarrow \text{SORT}(\text{verticalSlice}, \text{"ASC"})$ 
24  end for
25  for  $i \leftarrow 0$  to  $n - 1$  do
26    if  $i \bmod 2 = 0$  then
27       $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
28    else
29       $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"DESC"})$ 
30    end if
31  end for
32   $M' \leftarrow \text{OETS}(M')$ 
33  return  $M'$ 
34 end function
```

After sort columns
(algorithm 55)

$$\begin{bmatrix} 0 & 4 & 1 & 5 \\ 2 & 6 & 3 & 11 \\ 7 & 9 & 8 & 12 \\ 10 & 14 & 13 & 15 \end{bmatrix}$$

3n-Sort of Schnorr and Shamir - Example

3n-Sort Example (6)

131/174

Algorithm 56 3n-Sort Algorithm

Input: $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```
1 function THREESORT(M)
2   for  $i \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
3     for  $j \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
4       block  $\leftarrow M[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1] \leftarrow \text{SORT}(\text{block}, \text{"ASC"})$ 
5        $M'[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1] \leftarrow \text{SORT}(\text{block}, \text{"ASC"})$ 
6     end for
7   end for
8   for  $i \leftarrow 0$  to  $n - 1$  do
9      $M' \leftarrow \text{kWAYUNSHUFFLE}(M[i, :], n^{1/4})$ 
10  end for
11  for  $i \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
12    for  $j \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
13      block  $\leftarrow M[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1]$ 
14       $M'[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1] \leftarrow \text{SORT}(\text{block}, \text{"ASC"})$ 
15    end for
16  end for
17  for  $j \leftarrow 0$  to  $n - 1$  do
18     $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
19  end for
20  for  $k \leftarrow 0$  to  $\frac{n^{1/4}}{2} - 1$  do
21     $j \leftarrow k \left( \frac{n^{1/4}}{2} + 1 \right)$ 
22    verticalSlice  $\leftarrow M'[:, j \dots j + n^{1/4}]$ 
23     $M'[:, j \dots j + n^{1/4}] \leftarrow \text{SORT}(\text{verticalSlice}, \text{"ASC"})$ 
24  end for
25  for  $i \leftarrow 0$  to  $n - 1$  do
26    if  $i \bmod 2 = 0$  then
27       $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
28    else
29       $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"DESC"})$ 
30    end if
31  end for
32   $M' \leftarrow \text{OETS}(M')$ 
33  return  $M'$ 
34 end function
```

After sort vertical slices
(algorithm 56)

$$\begin{bmatrix} 0 & 2 & 1 & 3 \\ 4 & 6 & 5 & 8 \\ 7 & 9 & 11 & 12 \\ 10 & 14 & 13 & 15 \end{bmatrix}$$

3n-Sort of Schnorr and Shamir - Example

3n-Sort Example (7)

132/174

Algorithm 57 3n-Sort Algorithm

Input:

$M \in \mathbb{Z}^{n \times n}$: unsorted matrix

Output:

$M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```
1 function THREESORT(M)
2   for  $i \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
3     for  $j \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
4       block  $\leftarrow M[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1]$ 
5        $M'[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1] \leftarrow \text{SORT}(\text{block}, \text{"ASC"})$ 
6     end for
7   end for
8   for  $i \leftarrow 0$  to  $n - 1$  do
9      $M' \leftarrow \text{kWayUnshuffle}(M[i, :], n^{1/4})$ 
10  end for
11  for  $i \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
12    for  $j \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
13      block  $\leftarrow M[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1]$ 
14       $M'[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1] \leftarrow \text{SORT}(\text{block}, \text{"ASC"})$ 
15    end for
16  end for
17  for  $j \leftarrow 0$  to  $n - 1$  do
18     $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
19  end for
20  for  $k \leftarrow 0$  to  $\frac{n^{1/4}}{2} - 1$  do
21     $j \leftarrow k \left( \frac{n^{1/4}}{2} + 1 \right)$ 
22    verticalSlice  $\leftarrow M'[:, j \dots j + n^{1/4}]$ 
23     $M'[:, j \dots j + n^{1/4}] \leftarrow \text{SORT}(\text{verticalSlice}, \text{"ASC"})$ 
24  end for
25  for  $i \leftarrow 0$  to  $n - 1$  do
26    if  $i \bmod 2 = 0$  then
27       $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
28    else
29       $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"DESC"})$ 
30    end if
31  end for
32   $M' \leftarrow \text{OETS}(M')$ 
33  return  $M'$ 
34 end function
```

After sort rows
(algorithm 57)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 8 & 6 & 5 & 4 \\ 7 & 9 & 11 & 12 \\ 15 & 14 & 13 & 10 \end{bmatrix}$$

3n-Sort of Schnorr and Shamir - Example

3n-Sort Example (8)

133/174

Algorithm 58 3n-Sort Algorithm

Input: $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```
1 function THREESORT(M)
2   for  $i \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
3     for  $j \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
4       block  $\leftarrow M[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1]$ 
5        $M'[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1] \leftarrow \text{SORT}(\text{block}, \text{"ASC"})$ 
6     end for
7   end for
8   for  $i \leftarrow 0$  to  $n - 1$  do
9      $M' \leftarrow \text{kWayUNSHUFFLE}(M[i, :], n^{1/4})$ 
10  end for
11  for  $i \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
12    for  $j \leftarrow 0$  to  $n - 1$  step  $n^{3/4}$  do
13      block  $\leftarrow M[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1]$ 
14       $M'[i \dots i + n^{3/4} - 1, j \dots j + n^{3/4} - 1] \leftarrow \text{SORT}(\text{block}, \text{"ASC"})$ 
15    end for
16  end for
17  for  $j \leftarrow 0$  to  $n - 1$  do
18     $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
19  end for
20  for  $k \leftarrow 0$  to  $\frac{n^{1/4}}{2} - 1$  do
21     $j \leftarrow k \left( \frac{n^{1/4}}{2} + 1 \right)$ 
22    verticalSlice  $\leftarrow M'[:, j \dots j + n^{1/4}]$ 
23     $M'[:, j \dots j + n^{1/4}] \leftarrow \text{SORT}(\text{verticalSlice}, \text{"ASC"})$ 
24  end for
25  for  $i \leftarrow 0$  to  $n - 1$  do
26    if  $i \bmod 2 = 0$  then
27       $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
28    else
29       $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"DESC"})$ 
30    end if
31  end for
32   $M' \leftarrow \text{OETS}(M')$ 
33  return  $M'$ 
34 end function
```

After $n^{\frac{3}{4}}$ steps of OETS
(algorithm 58)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 7 & 6 & 5 & 4 \\ 8 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

- 1 Introduction
- 2 LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 3 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 4 Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 5 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 6 2D Odd-Even Transposition Sort**
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 7 Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 8 Conclusion
- 9 References

The 2D odd-even transposition sort is the simplest algorithm for sorting two-dimensional arrays, but it is very slow, time complexity $\Theta(n^2)$.

The idea of odd-even transposition sort [1] is generalized to two-dimensional arrays in a straightforward way. The elements are not just compared with their right and left neighbours, but also with their upper and lower neighbours.

The sorting direction in the array is snake-like.

- 1 Introduction
- 2 LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 3 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 4 Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 5 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 6 2D Odd-Even Transposition Sort
 - **Algorithm**
 - Correctness
 - Analysis
 - Example
- 7 Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 8 Conclusion
- 9 References

Algorithm 59 2D Odd-Even Transposition Sort Algorithm

```

Input:      20
     $M \in \mathbb{Z}^{n \times n}$ : unsorted matrix      21
Output:     22
     $M' \in \mathbb{Z}^{n \times n}$ : snake-sorted matrix  23

1 function TWOODEVENTRANSPOSITIONSORT(M)      24
2   while swapTakePlace do                    25
3     swapTakePlace  $\leftarrow$  false           26
4     for  $i \leftarrow 0$  to  $n - 1$  do           27
5       for  $j \leftarrow 1$  to  $n - 2$  step 2 do  28
6         if  $i \bmod 2 = 0 \wedge M[i][j] > M[i][j + 1]$  then  29
7           swap  $M[i][j]$  and  $M[i][j + 1]$       30
8           swapTakePlace  $\leftarrow$  true        31
9         else if  $i \bmod 2 = 1 \wedge M[i][j + 1] > M[i][j]$  then  32
10          swap  $M[i][j]$  and  $M[i][j + 1]$       33
11          swapTakePlace  $\leftarrow$  true        34
12        end if                               35
13      end for                               36
14    end for                               37
15    for  $i \leftarrow 0$  to  $n - 1$  do           38
16      for  $j \leftarrow 0$  to  $n - 2$  step 2 do  39
17        if  $i \bmod 2 = 0 \wedge M[i][j] > M[i][j + 1]$  then  40
18          swap  $M[i][j]$  and  $M[i][j + 1]$       41
19          swapTakePlace  $\leftarrow$  true        42
20        end if                               43
21      end for                               44
22    end while
23    return  $M'$ 
24  end function

```

- 1 Introduction
- 2 LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 3 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 4 Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 5 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 6 2D Odd-Even Transposition Sort
 - Algorithm
 - **Correctness**
 - Analysis
 - Example
- 7 Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 8 Conclusion
- 9 References

In two-dimensional odd-even transposition sort, the elements are not just compared with their right and left neighbours, but also with their upper and lower neighbours.

The four steps of two-dimensional odd-even transposition sort are repeated in round-robin order.

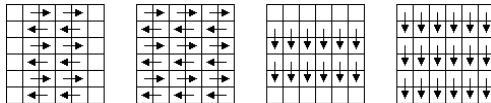


Figure: 2D OETS Steps

In each step, comparisons occur between pairs of adjacent cells:

- *rows*: comparisons between horizontal neighbours;
- *columns*: comparisons between vertical neighbours.

If two cells have the values 1 and 0, swapping puts them in the correct position according to the direction of comparison.

- Rows are neatly “cleaned” with row steps: the 1s move toward the final direction of the snake;
- columns help propagate the 1s downward, to the next rows in the snake.

After a sufficient number of iterations, the 1s “slide” like grains of sand along the surface of the array following the snake path. Eventually, all the 1’s will be at the bottommost positions in the snake-like linear sequence.

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ $3n$ -Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - **Analysis**
 - Example
- ⑦ Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑧ Conclusion
- ⑨ References

The 2D odd-even transposition sort is the simplest algorithm for sorting two-dimensional arrays, but it is very slow, time complexity $\Theta(n^2)$.

2D Odd-Even Transposition Sort Analysis (2)

If we ask what the constants of $\Theta(n^2)$ are, here is what we can say

$$T(n) = c \cdot n^2$$

where c is the constant factor depending on

- how many iterations are needed to fully sort in the worst case, but this is not fixed and depends on input;
- each iteration consist of
 - ▶ an odd step on rows;
 - ▶ an even step on rows;
 - ▶ an odd step on columns;
 - ▶ an even step on columns;

If we assume k full rounds are needed, then:

$$T(n) = 4k \cdot n^2$$

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - **Example**
- ⑦ Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑧ Conclusion
- ⑨ References

Algorithm 60 2D Odd-Even Transposition Sort Algorithm

```

Input:
  M ∈ ℤn×n: unsorted matrix
Output:
  M' ∈ ℤn×n: snake-sorted matrix

1 function TWOODODEVENTRANSPOSITION-
  SORT(M)
2   while swapTakePlace do
3     swapTakePlace ← false
4     for i ← 0 to n-1 do
5       for j ← 1 to n-2 step 2 do
6         if i mod 2 = 0 ∧ M[i][j] >
M[i][j+1] then
7           swap M[i][j] and M[i][j+1]
8           swapTakePlace ← true
9         else if i mod 2 = 1 ∧ M[i][j+1] >
M[i][j] then
10            swap M[i][j] and M[i][j+1]
11            swapTakePlace ← true
12          end if
13        end for
14      end for
15      for i ← 0 to n-1 do
16        for j ← 0 to n-2 step 2 do
17          if i mod 2 = 0 ∧ M[i][j] >
M[i][j+1] then
18            swap M[i][j] and M[i][j+1]
19            swapTakePlace ← true
20          else if i mod 2 = 1 ∧ M[i][j+1] >
M[i][j] then
21            swap M[i][j] and M[i][j+1]
22            swapTakePlace ← true
23          end if
24        end for
25      end for
26      for j ← 0 to n-1 do
27        for i ← 1 to n-2 step 2 do
28          if M[i][j] > M[i+1][j] then
29            swap M[i][j] and M[i+1][j]
30            swapTakePlace ← true
31          end if
32        end for
33      end for
34      for j ← 0 to n-1 do
35        for i ← 0 to n-2 step 2 do
36          if M[i][j] > M[i+1][j] then
37            swap M[i][j] and M[i+1][j]
38            swapTakePlace ← true
39          end if
40        end for
41      end for
42    end while
43    return M'
44 end function

```

Initial situation

$$\begin{bmatrix} 0 & 1 & 7 & 12 \\ 4 & 5 & 9 & 8 \\ 13 & 6 & 15 & 14 \\ 3 & 2 & 11 & 10 \end{bmatrix}$$

2D Odd-Even Transposition Sort Example (2)

Algorithm 61 2D Odd-Even Transposition Sort Algorithm

```

Input:      M ∈ ℤn×n: unsorted matrix
Output:     M' ∈ ℤn×n: snake-sorted matrix

1 function TWOODODEVENTRANSPOSITION-
  SORT(M)
2   while swapTakePlace do
3     swapTakePlace ← false
4     for i ← 0 to n - 1 do
5       for j ← 1 to n - 2 step 2 do
6         if i mod 2 = 0 ∧ M[i][j] >
          M[i][j + 1] then
7           swap M[i][j] and M[i][j + 1]
8           swapTakePlace ← true
9         else if i mod 2 = 1 ∧ M[i][j] +
          1] > M[i][j] then
10          swap M[i][j] and M[i][j + 1]
11          swapTakePlace ← true
12        end if
13      end for
14    end for
15    for i ← 0 to n - 1 do
16      for j ← 0 to n - 2 step 2 do
17        if i mod 2 = 0 ∧ M[i][j] >
          M[i][j + 1] then
18          swap M[i][j] and M[i][j + 1]
19          swapTakePlace ← true
20        else if i mod 2 = 1 ∧ M[i][j] +
          1] > M[i][j] then
21          swap M[i][j] and M[i][j + 1]
22          swapTakePlace ← true
23        end if
24      end for
25    end for
26    for j ← 0 to n - 1 do
27      for i ← 1 to n - 2 step 2 do
28        if M[i][j] > M[i + 1][j] then
29          swap M[i][j] and M[i + 1][j]
30          swapTakePlace ← true
31        end if
32      end for
33    end for
34    for j ← 0 to n - 1 do
35      for i ← 0 to n - 2 step 2 do
36        if M[i][j] > M[i + 1][j] then
37          swap M[i][j] and M[i + 1][j]
38          swapTakePlace ← true
39        end if
40      end for
41    end for
42  end while
43  return M'
44 end function

```

After odd oets step to the rows (algorithm 61)

$$\begin{bmatrix} 0 & 1 & 7 & 12 \\ 4 & 9 & 5 & 8 \\ 13 & 6 & 15 & 14 \\ 3 & 11 & 2 & 10 \end{bmatrix}$$

After even oets step to the rows (algorithm 61)

$$\begin{bmatrix} 0 & 1 & 7 & 12 \\ 9 & 4 & 8 & 5 \\ 6 & 13 & 14 & 15 \\ 11 & 3 & 10 & 2 \end{bmatrix}$$

After odd oets step to the columns (algorithm 61)

$$\begin{bmatrix} 0 & 1 & 7 & 12 \\ 6 & 4 & 8 & 5 \\ 9 & 13 & 14 & 15 \\ 11 & 3 & 10 & 2 \end{bmatrix}$$

After even oets step to the columns (algorithm 61)

$$\begin{bmatrix} 0 & 1 & 7 & 5 \\ 6 & 4 & 8 & 12 \\ 9 & 3 & 10 & 2 \\ 11 & 13 & 14 & 15 \end{bmatrix}$$

Algorithm 62 2D Odd-Even Transposition Sort Algorithm

```

Input:       $M \in \mathbb{Z}^{n \times n}$ : unsorted matrix
Output:      $M' \in \mathbb{Z}^{n \times n}$ : snake-sorted matrix

1 function TWOODODEVENTRANSPOSITION-
  SORT(M)
2   while swapTakePlace do
3     swapTakePlace  $\leftarrow$  false
4     for  $i \leftarrow 0$  to  $n-1$  do
5       for  $j \leftarrow 1$  to  $n-2$  step 2 do
6         if  $i \bmod 2 = 0 \wedge M[i][j] >$ 
            $M[i][j+1]$  then
7           swap  $M[i][j]$  and  $M[i][j+1]$ 
8           swapTakePlace  $\leftarrow$  true
9         else if  $i \bmod 2 = 1 \wedge M[i][j+1]$ 
            $> M[i][j]$  then
10          swap  $M[i][j]$  and  $M[i][j+1]$ 
11          swapTakePlace  $\leftarrow$  true
12        end if
13      end for
14    end for
15    for  $i \leftarrow 0$  to  $n-1$  do
16      for  $j \leftarrow 0$  to  $n-2$  step 2 do
17        if  $i \bmod 2 = 0 \wedge M[i][j] >$ 
           $M[i][j+1]$  then
18          swap  $M[i][j]$  and  $M[i][j+1]$ 
19        else if  $i \bmod 2 = 1 \wedge M[i][j+1]$ 
           $> M[i][j]$  then
20          swap  $M[i][j]$  and  $M[i][j+1]$ 
21          swapTakePlace  $\leftarrow$  true
22        end if
23      end for
24    end for
25  end while
26  for  $j \leftarrow 0$  to  $n-1$  do
27    for  $i \leftarrow 1$  to  $n-2$  step 2 do
28      if  $M[i][j] > M[i+1][j]$  then
29        swap  $M[i][j]$  and  $M[i+1][j]$ 
30        swapTakePlace  $\leftarrow$  true
31      end if
32    end for
33  end for
34  for  $j \leftarrow 0$  to  $n-1$  do
35    for  $i \leftarrow 0$  to  $n-2$  step 2 do
36      if  $M[i][j] > M[i+1][j]$  then
37        swap  $M[i][j]$  and  $M[i+1][j]$ 
38        swapTakePlace  $\leftarrow$  true
39      end if
40    end for
41  end for
42 end while
43 return  $M'$ 
44 end function

```

After odd oets step to the rows (algorithm 62)

$$\begin{bmatrix} 0 & 1 & 7 & 5 \\ 6 & 8 & 4 & 12 \\ 9 & 3 & 10 & 2 \\ 11 & 14 & 13 & 15 \end{bmatrix}$$

After even oets step to the rows (algorithm 62)

$$\begin{bmatrix} 0 & 1 & 5 & 7 \\ 8 & 6 & 12 & 4 \\ 3 & 9 & 2 & 10 \\ 14 & 11 & 15 & 13 \end{bmatrix}$$

After odd oets step to the columns (algorithm 62)

$$\begin{bmatrix} 0 & 1 & 5 & 7 \\ 3 & 6 & 2 & 4 \\ 8 & 9 & 12 & 10 \\ 14 & 11 & 15 & 13 \end{bmatrix}$$

After even oets step to the columns (algorithm 62)

$$\begin{bmatrix} 0 & 1 & 2 & 4 \\ 3 & 6 & 5 & 7 \\ 8 & 9 & 12 & 10 \\ 14 & 11 & 15 & 13 \end{bmatrix}$$

2D Odd-Even Transposition Sort Example (4)

Algorithm 63 2D Odd-Even Transposition Sort Algorithm

```

Input:       $M \in \mathbb{Z}^{n \times n}$ : unsorted matrix
Output:      $M' \in \mathbb{Z}^{n \times n}$ : snake-sorted matrix

1 function TWOODODEVENTRANSPOSITION-
  SORT(M)
2   while swapTakePlace do
3     swapTakePlace  $\leftarrow$  false
4     for  $i \leftarrow 0$  to  $n-1$  do
5       for  $j \leftarrow 1$  to  $n-2$  step 2 do
6         if  $i \bmod 2 = 0 \wedge M[i][j] >$ 
            $M[i][j+1]$  then
7           swap  $M[i][j]$  and  $M[i][j+1]$ 
8           swapTakePlace  $\leftarrow$  true
9         else if  $i \bmod 2 = 1 \wedge M[i][j+1]$ 
            $> M[i][j]$  then
10          swap  $M[i][j]$  and  $M[i][j+1]$ 
11          swapTakePlace  $\leftarrow$  true
12        end if
13      end for
14    end for
15    for  $i \leftarrow 0$  to  $n-1$  do
16      for  $j \leftarrow 0$  to  $n-2$  step 2 do
17        if  $i \bmod 2 = 0 \wedge M[i][j] >$ 
           $M[i][j+1]$  then
18          swap  $M[i][j]$  and  $M[i][j+1]$ 
19        else if  $i \bmod 2 = 1 \wedge M[i][j+1]$ 
           $> M[i][j]$  then
20          swap  $M[i][j]$  and  $M[i][j+1]$ 
21          swapTakePlace  $\leftarrow$  true
22        end if
23      end for
24    end for
25  end while
26  for  $j \leftarrow 0$  to  $n-1$  do
27    for  $i \leftarrow 1$  to  $n-2$  step 2 do
28      if  $M[i][j] > M[i+1][j]$  then
29        swap  $M[i][j]$  and  $M[i+1][j]$ 
30        swapTakePlace  $\leftarrow$  true
31      end if
32    end for
33  end for
34  for  $j \leftarrow 0$  to  $n-1$  do
35    for  $i \leftarrow 0$  to  $n-2$  step 2 do
36      if  $M[i][j] > M[i+1][j]$  then
37        swap  $M[i][j]$  and  $M[i+1][j]$ 
38        swapTakePlace  $\leftarrow$  true
39      end if
40    end for
41  end for
42 end while
43 return  $M'$ 
44 end function

```

After odd oets step to the rows (algorithm 63)

$$\begin{bmatrix} 0 & 1 & 2 & 4 \\ 3 & 6 & 5 & 7 \\ 8 & 9 & 12 & 10 \\ 14 & 15 & 11 & 13 \end{bmatrix}$$

After even oets step to the rows (algorithm 63)

$$\begin{bmatrix} 0 & 1 & 2 & 4 \\ 6 & 3 & 7 & 5 \\ 8 & 9 & 10 & 12 \\ 15 & 14 & 13 & 11 \end{bmatrix}$$

After odd oets step to the columns (algorithm 63)

$$\begin{bmatrix} 0 & 1 & 2 & 4 \\ 6 & 3 & 7 & 5 \\ 8 & 9 & 10 & 12 \\ 15 & 14 & 13 & 11 \end{bmatrix}$$

After even oets step to the columns (algorithm 63)

$$\begin{bmatrix} 0 & 1 & 2 & 4 \\ 6 & 3 & 7 & 5 \\ 8 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

Algorithm 64 2D Odd-Even Transposition Sort Algorithm

```

Input:       $M \in \mathbb{Z}^{n \times n}$ : unsorted matrix
Output:      $M' \in \mathbb{Z}^{n \times n}$ : snake-sorted matrix

1 function TWOODODEVENTRANSPOSITION-
  SORT( $M$ )
2   while swapTakePlace do
3     swapTakePlace  $\leftarrow$  false
4     for  $i \leftarrow 0$  to  $n-1$  do
5       for  $j \leftarrow 1$  to  $n-2$  step 2 do
6         if  $i \bmod 2 = 0 \wedge M[i][j] >$ 
            $M[i][j+1]$  then
7           swap  $M[i][j]$  and  $M[i][j+1]$ 
8           swapTakePlace  $\leftarrow$  true
9         else if  $i \bmod 2 = 1 \wedge M[i][j+1]$ 
            $> M[i][j]$  then
10          swap  $M[i][j]$  and  $M[i][j+1]$ 
11          swapTakePlace  $\leftarrow$  true
12        end if
13      end for
14    end for
15    for  $i \leftarrow 0$  to  $n-1$  do
16      for  $j \leftarrow 0$  to  $n-2$  step 2 do
17        if  $i \bmod 2 = 0 \wedge M[i][j] >$ 
           $M[i][j+1]$  then
18          swap  $M[i][j]$  and  $M[i][j+1]$ 
19        else if  $i \bmod 2 = 1 \wedge M[i][j+1]$ 
           $> M[i][j]$  then
20          swap  $M[i][j]$  and  $M[i][j+1]$ 
21          swapTakePlace  $\leftarrow$  true
22        end if
23      end for
24    end for
25  end while
26  for  $j \leftarrow 0$  to  $n-1$  do
27    for  $i \leftarrow 1$  to  $n-2$  step 2 do
28      if  $M[i][j] > M[i+1][j]$  then
29        swap  $M[i][j]$  and  $M[i+1][j]$ 
30        swapTakePlace  $\leftarrow$  true
31      end if
32    end for
33  end for
34  for  $j \leftarrow 0$  to  $n-1$  do
35    for  $i \leftarrow 0$  to  $n-2$  step 2 do
36      if  $M[i][j] > M[i+1][j]$  then
37        swap  $M[i][j]$  and  $M[i+1][j]$ 
38        swapTakePlace  $\leftarrow$  true
39      end if
40    end for
41  end for
42 end while
43 return  $M'$ 
44 end function

```

After odd oets step to the rows (algorithm 64)

$$\begin{bmatrix} 0 & 1 & 2 & 4 \\ 6 & 7 & 3 & 5 \\ 8 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

After even oets step to the rows (algorithm 64)

$$\begin{bmatrix} 0 & 1 & 2 & 4 \\ 7 & 6 & 5 & 3 \\ 8 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

After odd oets step to the columns (algorithm 64)

$$\begin{bmatrix} 0 & 1 & 2 & 4 \\ 7 & 6 & 5 & 3 \\ 8 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

After even oets step to the columns (algorithm 64)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 7 & 6 & 5 & 4 \\ 8 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ **Shearsort**
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑧ Conclusion
- ⑨ References

The shearsort [4] is a very simple algorithm for sorting two-dimensional arrays. It just sorts the rows and the columns of the array in turn.

The sorting direction in the array is snake-like.

- 1 Introduction
- 2 LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 3 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 4 Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- 5 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 6 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- 7 **Shearsort**
 - **Algorithm**
 - Correctness
 - Analysis
 - Example
- 8 Conclusion
- 9 References

Algorithm 65 Shearsort Algorithm

Input: $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```
1 function SHEARSORT(M)
2   for  $k \leftarrow 0$  to  $\log n - 1$  do
3     for  $i \leftarrow 0$  to  $n - 1$  do
4        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"ASC"})$ 
5     end for
6     for  $j \leftarrow 0$  to  $n - 1$  do
7        $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
8     end for
9      $k \leftarrow k + 1$ 
10  end for
11  for  $i \leftarrow 0$  to  $n - 1$  do
12     $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
13  end for
14 end function
```

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ **Shearsort**
 - Algorithm
 - **Correctness**
 - Analysis
 - Example
- ⑧ Conclusion
- ⑨ References

After sorting the rows in the first step, there are $\frac{n}{2}$ rows that are sorted from left to right and $\frac{n}{2}$ rows that are sorted from right to left. When sorting the columns, every two of these rows are combined to at least one clean row.

After the first iteration the array consist of some clean 0-rows, some clean 1-rows and an unsorted zone in between consisting of at most $\frac{n}{2}$ rows.

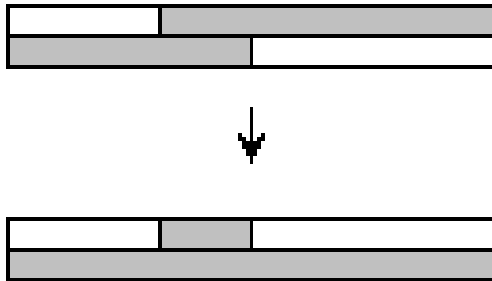


Figure: First Iteration

In the second iteration, every two rows of the unsorted zone are again combined to at least one clean row. Thus, after the second iteration the unsorted zone consists of at most $\frac{n}{4}$ rows, and so on.

In the second iteration, every two rows of the unsorted zone are again combined to at least one clean row. Thus, after the second iteration the unsorted zone consists of at most $\frac{n}{4}$ rows, and so on.

After $\log n$ steps, there is at most one dirty row. In the additional last step this row is sorted, and the whole array is sorted.

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ **Shearsort**
 - Algorithm
 - Correctness
 - **Analysis**
 - Example
- ⑧ Conclusion
- ⑨ References

Sort a n -length rows takes n steps. The algorithm performs $\log n$ iteration, so it requires $n \log n$ steps for sorting the rows.

In each iteration, the height of the unsorted zone decreases by a factor of 2. This means that the columns contain an unsorted zone of decreasing length. So, the number of steps to sort the column is

$$n + \frac{n}{2} + \frac{n}{4} + \dots + 2 = 2n - 2$$

In the additional last step this row is sorted, and the whole array is sorted. So, the final complexity is $T(n^2) = n \log_2 n + 3n - 2$

The number of comparator required is

$$\begin{aligned} C(n^2) &= T(n^2) \frac{n(n-1)}{2} = \\ &= \frac{n^3 \log_2 n}{2} + \frac{3}{2} n^3 - \frac{n^2 \log_2 n}{2} - \frac{5}{2} n^2 + n \end{aligned}$$

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ 3n-Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ **Shearsort**
 - Algorithm
 - Correctness
 - Analysis
 - **Example**
- ⑧ Conclusion
- ⑨ References

Algorithm 66 Shearsort Algorithm

Input: $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```
1 function SHEARSORT(M)
2   for  $k \leftarrow 0$  to  $\log n - 1$  do
3     for  $i \leftarrow 0$  to  $n - 1$  do
4        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"ASC"})$ 
5     end for
6     for  $j \leftarrow 0$  to  $n - 1$  do
7        $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
8     end for
9      $k \leftarrow k + 1$ 
10  end for
11  for  $i \leftarrow 0$  to  $n - 1$  do
12     $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
13  end for
14 end function
```

Initial situation

$$\begin{bmatrix} 0 & 1 & 7 & 12 \\ 4 & 5 & 9 & 8 \\ 13 & 6 & 15 & 14 \\ 3 & 2 & 11 & 10 \end{bmatrix}$$

Algorithm 67 Shearsort Algorithm**Input:** $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```

1 function SHEARSORT(M)
2   for  $k \leftarrow 0$  to  $\log n - 1$  do
3     for  $i \leftarrow 0$  to  $n - 1$  do
4        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"ASC"})$ 
5     end for
6     for  $j \leftarrow 0$  to  $n - 1$  do
7        $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
8     end for
9      $k \leftarrow k + 1$ 
10  end for
11  for  $i \leftarrow 0$  to  $n - 1$  do
12     $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
13  end for
14 end function

```

After sort rows (algorithm 67)

$$\begin{bmatrix} 0 & 1 & 7 & 12 \\ 9 & 8 & 5 & 4 \\ 6 & 13 & 14 & 15 \\ 11 & 10 & 3 & 2 \end{bmatrix}$$

After sort columns (algorithm 67)

$$\begin{bmatrix} 0 & 1 & 3 & 2 \\ 6 & 8 & 5 & 4 \\ 9 & 10 & 7 & 12 \\ 11 & 13 & 14 & 15 \end{bmatrix}$$

Algorithm 68 Shearsort Algorithm

Input: $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```
1 function SHEARSORT(M)
2   for  $k \leftarrow 0$  to  $\log n - 1$  do
3     for  $i \leftarrow 0$  to  $n - 1$  do
4        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"ASC"})$ 
5     end for
6     for  $j \leftarrow 0$  to  $n - 1$  do
7        $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
8     end for
9      $k \leftarrow k + 1$ 
10  end for
11  for  $i \leftarrow 0$  to  $n - 1$  do
12     $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
13  end for
14 end function
```

After sort rows (algorithm 68)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 8 & 6 & 5 & 4 \\ 7 & 9 & 10 & 12 \\ 15 & 14 & 13 & 11 \end{bmatrix}$$

After sort columns (algorithm 68)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 7 & 6 & 5 & 4 \\ 8 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

Algorithm 69 Shearsort Algorithm

Input: $M \in \mathbb{Z}^{n \times n}$: unsorted matrix**Output:** $M' \in \mathbb{Z}^{n \times n}$: snake-sorted matrix

```
1 function SHEARSORT(M)
2   for  $k \leftarrow 0$  to  $\log n - 1$  do
3     for  $i \leftarrow 0$  to  $n - 1$  do
4        $M'[i, :] \leftarrow \text{SORT}(M[i, :], \text{"ASC"})$ 
5     end for
6     for  $j \leftarrow 0$  to  $n - 1$  do
7        $M'[:, j] \leftarrow \text{SORT}(M'[:, j], \text{"ASC"})$ 
8     end for
9      $k \leftarrow k + 1$ 
10  end for
11  for  $i \leftarrow 0$  to  $n - 1$  do
12     $M'[i, :] \leftarrow \text{SORT}(M'[i, :], \text{"ASC"})$ 
13  end for
14 end function
```

After last sort rows (algorithm 69)

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 7 & 6 & 5 & 4 \\ 8 & 9 & 10 & 11 \\ 15 & 14 & 13 & 12 \end{bmatrix}$$

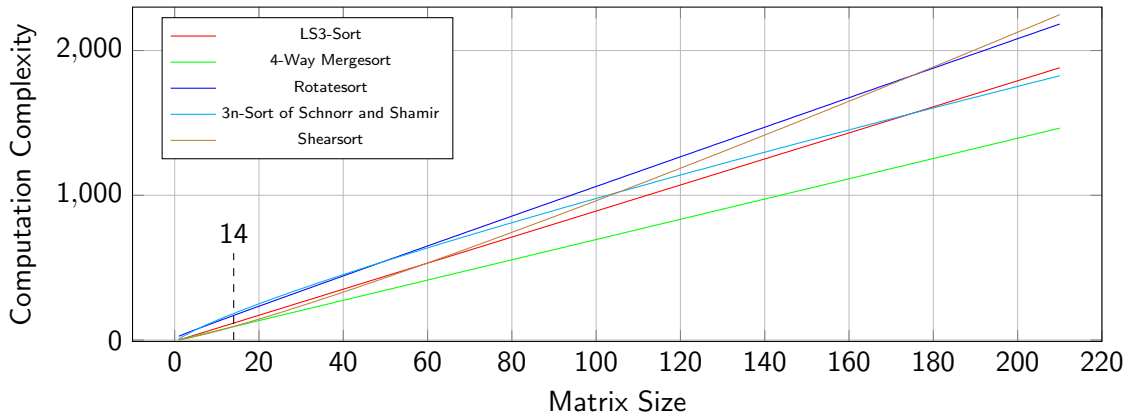
- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ $3n$ -Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑧ **Conclusion**
- ⑨ References

Let us consider having a two-dimensional array $M \in \mathbb{Z}^{n \times n}$

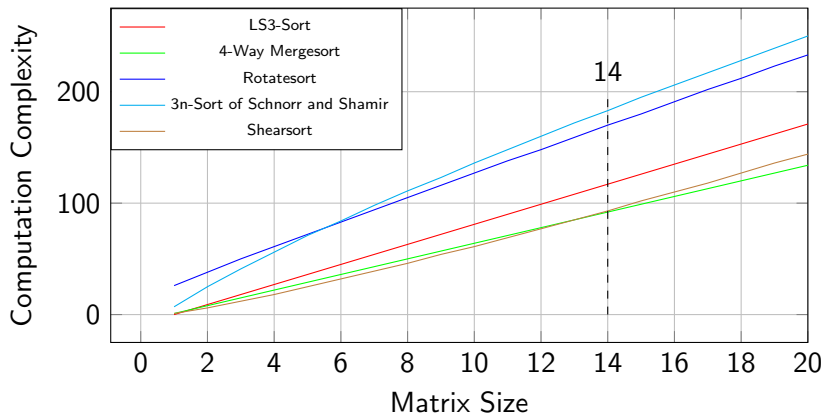
Sorting Network	Complexity $T(n^2)$
LS3-Sort [2]	$9n - 9$
4-Way Mergesort [5]	$7n - 6$
Rotatesort [3]	$10n + 5\sqrt{n} + 11$
3n-Sort of Schnorr and Shamir [6]	$3n + 22n^{\frac{3}{4}} - 18$
2D Odd-Even Transposition Sort [1]	$\Theta(n^2)$
Shearsort [4]	$n \log_2 n + 3n - 2$

Table: Summary of Sorting Network Complexity

Matrix Size - Complexity



Matrix Size - Complexity



It can be deduced from the graph that for:

- $n < 14$ the best sorting network is **Shearsort**;
- $n \geq 14$ the best sorting network is **4-way mergesort**

- ① Introduction
- ② LS3-Sort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ③ 4-Way Mergesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ④ Rotatesort
 - Algorithms
 - Correctness
 - Analysis
 - Example
- ⑤ $3n$ -Sort of Schnorr and Shamir
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑥ 2D Odd-Even Transposition Sort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑦ Shearsort
 - Algorithm
 - Correctness
 - Analysis
 - Example
- ⑧ Conclusion
- ⑨ **References**

- [1] Donald E. Knuth. *The Art of Computer Programming, Volume III: Sorting and Searching*. Addison-Wesley, 1973. ISBN: 0-201-03803-X.
- [2] Hans-Werner Lang et al. "Systolic Sorting on a Mesh-Connected Network". In: *IEEE Trans. Computers* 34.7 (1985), pp. 652–658. DOI: 10.1109/TC.1985.1676603. URL: <https://doi.org/10.1109/TC.1985.1676603>.
- [3] John M. Marberg and Eli Gafni. "Sorting in Constant Number of Row and Column Phases on a Mesh". In: *Algorithmica* 3 (1988), pp. 561–572. DOI: 10.1007/BF01762132. URL: <https://doi.org/10.1007/BF01762132>.
- [4] Isaac D. Scherson and Sandeep Sen. "Parallel Sorting in Two-Dimensional VLSI Models of Computation". In: *IEEE Trans. Computers* 38.2 (1989), pp. 238–249. DOI: 10.1109/12.16500. URL: <https://doi.org/10.1109/12.16500>.

- [5] Manfred Schimpler. *Fast Sorting on the Instruction Systolic Array*. Bericht 8709. Institut für Informatik, Christian-Albrechts-Universität Kiel, 1987.
- [6] Claus-Peter Schnorr and Adi Shamir. “An Optimal Sorting Algorithm for Mesh Connected Computers”. In: *Proceedings of the 18th Annual ACM Symposium on Theory of Computing, May 28-30, 1986, Berkeley, California, USA*. Ed. by Juris Hartmanis. ACM, 1986, pp. 255–263. DOI: 10.1145/12130.12156. URL: <https://doi.org/10.1145/12130.12156>.